Artículo de investigación aplicada

Detección de spam en mensajes SMS mediante aprendizaje supervisado y revisión humana de predicciones inciertas

SMS spam detection through supervised learning and human review of uncertain predictions

Baca Flores, Flavio Manuel André
Universidad Privada del Norte, Perú
Correo: **[POR COMPLETAR]**
ORCID: **[POR COMPLETAR]**

Espinoza Marquez, Adriano Felipe
Universidad Privada del Norte, Perú
Correo: **[POR COMPLETAR]**
ORCID: **[POR COMPLETAR]**

Goray Gonzales, Nicolás Alexander
Universidad Privada del Norte, Perú
Correo: **[POR COMPLETAR]**
ORCID: **[POR COMPLETAR]**

Paredes Moya, Pablo Sebastián
Universidad Privada del Norte, Perú
Correo: **[POR COMPLETAR]**
ORCID: **[POR COMPLETAR]**

Grupo 03 · Sede Trujillo – San Isidro, Perú
Curso: Sistemas Inteligentes y Machine Learning
Periodo académico 2026-2 · Proyecto de ocho semanas
Estado: investigación en curso (entrega de avance)

Resumen

El spam por mensajes de texto (SMS) es un canal persistente de publicidad no solicitada y de fraude: en Estados Unidos las pérdidas reportadas por estafas mediante mensajes de texto alcanzaron 470 millones de dólares en 2024 y, en el Perú, las denuncias por fraude informático aumentaron 58 % en 2023. Este artículo presenta una investigación aplicada, todavía en curso, que construye y evalúa un detector de spam SMS que envía a revisión humana las predicciones inciertas. Se utilizó el corpus SMS Spam Collection v.1 del repositorio UCI (5574 mensajes en inglés; 13.4 % spam) con una partición 60/20/20 estratificada y agrupada, de modo que los 403 duplicados exactos no cruzaran particiones. Se compararon la regresión logística y la máquina de vectores de soporte (SVM) lineal sobre representaciones TF-IDF; los hiperparámetros se eligieron por validación cruzada agrupada, la SVM se calibró con la sigmoide de Platt y el umbral de confianza τ se fijó en validación antes de abrir la partición de prueba. La SVM lineal calibrada (C = 10, unigramas y bigramas) obtuvo en prueba una precision de spam de 0.986 (IC 95 % bootstrap: 0.963–1.000), un recall de 0.913 y un F1 de 0.948, valores que superan las metas fijadas de antemano (F1 ≥ 0.90; precision ≥ 0.95). Con τ = 0.70, el 0.54 % de los mensajes pasó a revisión y el error entre los mensajes resueltos automáticamente bajó de 1.35 % a 0.99 %; no obstante, 11 de los 13 falsos negativos se aceptaron con confianza alta, de modo que la abstención no corrige los errores emitidos con seguridad. Se implementó un prototipo web con Flask y SQLite que clasifica, se abstiene y registra revisiones, sin reentrenamiento. Los resultados no permiten afirmar validez para mensajes en español ni para patrones de fraude recientes.

Palabras clave: detección de spam, mensajes SMS, aprendizaje supervisado, clasificación selectiva, revisión humana.

Abstract

Text-message (SMS) spam remains a persistent channel for unsolicited advertising and fraud: in the United States, reported losses to text scams reached USD 470 million in 2024, and in Peru complaints of computer fraud rose by 58 % in 2023. This article reports ongoing applied research that builds and evaluates an SMS spam detector that routes uncertain predictions to human review. The SMS Spam Collection v.1 corpus from the UCI repository (5,574 English messages; 13.4 % spam) was split 60/20/20 with stratification and grouping, so that the 403 exact duplicates could not cross partitions. Logistic regression and a linear support vector machine (SVM) were compared on TF-IDF representations; hyperparameters were selected by grouped cross-validation, the SVM was calibrated with Platt's sigmoid, and the confidence threshold τ was fixed on validation data before the test partition was opened. The calibrated linear SVM (C = 10, unigrams and bigrams) achieved a test spam precision of 0.986 (95 % bootstrap CI: 0.963–1.000), recall of 0.913 and F1 of 0.948, exceeding the targets set in advance (F1 ≥ 0.90; precision ≥ 0.95). With τ = 0.70, 0.54 % of the messages were sent to review and the error among automatically resolved messages fell from 1.35 % to 0.99 %; however, 11 of the 13 false negatives were accepted with high confidence, so abstention does not correct confident errors. A Flask and SQLite web prototype that classifies, abstains and records reviews was implemented, without retraining. The results do not support claims of validity for Spanish-language messages or for recent fraud patterns.

Keywords: spam detection, SMS messages, supervised learning, selective classification, human review.

I. INTRODUCCIÓN

El servicio de mensajes cortos (SMS) sigue siendo un canal masivo y barato para comunicaciones legítimas —alertas bancarias, códigos de verificación, recordatorios— y, por la misma razón, para el envío de mensajes no solicitados y fraudulentos. La Comisión Federal de Comercio de Estados Unidos informó que las pérdidas reportadas por estafas mediante mensajes de texto llegaron a 470 millones de dólares en 2024, más de cinco veces la cifra de 2020, aun cuando el número de reportes disminuyó; las cinco modalidades más frecuentes fueron falsos problemas de entrega de paquetes, ofertas de empleo fraudulentas, falsas alertas de fraude bancario, cobros de peaje inexistentes y mensajes de «número equivocado» (Federal Trade Commission [FTC], 2025). En la literatura técnica, el filtrado de spam SMS se aborda como un problema de clasificación supervisada de textos cortos, con resultados altos sobre corpus de referencia (Ghourabi y Alohaly, 2023; Şahin et al., 2026), pero también con advertencias sobre la capacidad de los emisores para evadir los filtros y sobre la caída del desempeño cuando cambia la fuente de los mensajes (Bani Younes y Ababneh, 2026; Salman et al., 2024).

En el Perú, el Ministerio Público registró 21 842 denuncias por fraude informático en 2023, un aumento de 58 % respecto del año anterior, y el fraude informático fue el delito informático más denunciado (Diario Oficial El Peruano, 2024). La prensa oficial describe el *smishing* como el envío de un SMS con un enlace a una página falsa que solicita datos personales y financieros, y recuerda que los bancos no solicitan claves por mensajes de texto (Diario Oficial El Peruano, 2023). Estas cifras abarcan todas las modalidades de fraude informático y no permiten aislar la fracción atribuible al spam SMS; no se localizó una estadística oficial reciente desagregada por canal, por lo que esa cuantificación queda como requisito pendiente.

En el ámbito local, Lulli Caceres y Ortiz Gutierrez (2024) analizaron el tratamiento del fraude informático en la Segunda Fiscalía Penal Corporativa de Trujillo durante 2023 e incluyeron un caso iniciado mediante un SMS con una oferta de préstamo bancario (Tabla 14, p. 51). Esta evidencia cualitativa complementa los once reclamos por estafas mediante mensajes de texto registrados en los primeros cuatro meses de 2011 (Redacción Correo, 2011), pero no estima la frecuencia actual del spam SMS en la región. Además, la relevancia local no implica representatividad del corpus: el conjunto empleado reúne mensajes en inglés recopilados hacia 2011 en el Reino Unido y Singapur (Almeida et al., 2011), por lo que el modelo no está validado para mensajes en español ni para los patrones de fraude actuales del país.

Los estudios recientes sobre este corpus informan métricas elevadas, pero rara vez explican cómo tratan los mensajes duplicados, si las probabilidades del modelo están calibradas o qué hacer con las predicciones dudosas. Los marcos de aprendizaje con opción de rechazo (Hendrickx et al., 2024) y de aprendizaje automático con intervención humana (Mosqueira-Rey et al., 2023) ofrecen una alternativa: el sistema decide automáticamente sólo cuando su confianza es suficiente y deriva el resto a una persona. Este trabajo combina ambos elementos en una aplicación web y evalúa el equilibrio entre los errores de las decisiones automáticas y la proporción de mensajes que el sistema puede resolver sin intervención.

1.1 Antecedentes

Para clasificar los antecedentes se aplicó un criterio geográfico explícito: son **internacionales** los estudios cuyos autores pertenecen a instituciones fuera del Perú; **nacionales**, los que tienen al menos una institución peruana fuera de La Libertad; y **locales**, los realizados por instituciones de Trujillo o La Libertad o con población de esa región. Se aceptaron publicaciones aparecidas desde noviembre de 2021, es decir, con una antigüedad no mayor de cinco años respecto de la entrega prevista en el ciclo 2026-2. La búsqueda se realizó el 18 de octubre de 2026 en buscadores académicos y generales, Crossref, Scimago Journal & Country Rank, el repositorio ALICIA de CONCYTEC y los sitios de las editoriales; el registro de búsqueda y la matriz completa de extracción se presentan en el Anexo A. Las métricas se tomaron de los artículos originales, indicando la tabla, figura o sección de procedencia; cuando un dato no figura en el artículo se indica «no reportado», y cuando no fue posible leer el texto completo se indica «no verificado».

Antecedentes internacionales

Ghourabi y Alohaly (2023), *Enhancing spam message classification and detection using transformer-based embedding and ensemble learning* (Arabia Saudita y Túnez). Su objetivo fue proponer un detector de spam SMS que combine representaciones preentrenadas basadas en transformadores con aprendizaje por conjuntos. Con un diseño experimental cuantitativo, limpiaron los textos, obtuvieron vectores de 1536 dimensiones con GPT-3, los compararon con TF-IDF, *word embeddings* y BERT en diez clasificadores, y combinaron SVM, KNN, CNN y LightGBM mediante votación ponderada con pesos ajustados por optimización bayesiana, usando una partición 80/20. La población fueron mensajes SMS en inglés y la muestra, los 5574 mensajes del SMS Spam Collection (4827 legítimos y 747 spam). El conjunto propuesto alcanzó accuracy 0.9991, precision 1.000, recall 0.9932 y F1 0.9966 (Tabla 3 del artículo); la SVM con TF-IDF obtuvo accuracy 0.9785 y con GPT-3, 0.9964 (Tabla 2). El tratamiento de duplicados es no reportado. Comparte nuestro corpus y respalda la SVM como candidata, pero usa una única partición aleatoria y representaciones de alto costo, sin abstención.

Salman et al. (2024), *Investigating evasive techniques in SMS spam filtering: A comparative analysis of machine learning models* (Australia), analizaron la evolución del spam SMS y compararon modelos y servicios frente a técnicas de evasión. Consolidaron y depuraron mensajes en inglés, compararon representaciones léxicas y semánticas y declararon una partición 80/20, además de 225 SMS spam separados para evaluación adversarial. El cuerpo y la Tabla 4 describen 67 018 mensajes (40 837 legítimos y 26 181 spam), aunque el resumen menciona más de 68 000. TCSVM con Word2Vec obtuvo precision 99 %, recall 98 %, accuracy 97.8 % y F1 99 % (Tabla 12, p. 24315). En el conjunto adversarial separado, ese modelo pasó de accuracy 83.1 % y F1 91 % sin modificación a 11.6 % y 21 % con inserción de espacios (Tabla 14, p. 24317). Estos resultados sustentan el riesgo de evasión dentro de su protocolo y no permiten inferir superioridad respecto de nuestro corpus.

Altunay y Albayrak (2024), *SMS spam detection system based on deep learning architectures for Turkish and English messages* (Turquía). Su objetivo fue proponer un modelo híbrido CNN + GRU para detectar spam en SMS en turco e inglés. Aplicaron limpieza, eliminación de palabras vacías, normalización morfológica y *embeddings* entrenados, y compararon CNN, GRU y CNN + GRU. La muestra combinó el TurkishSMSCollection (4751 mensajes) y el SMS Spam Collection (5574), en total 10 325 mensajes; el conjunto de prueba tuvo 965 mensajes no spam y 665 spam. Con el modelo híbrido informaron accuracy 0.9908, precision 0.9907, recall 0.9938 y F1 0.9922 sobre los datos combinados (sección 5), con 956 de 965 y 659 de 665 mensajes correctamente clasificados (Figura 6). Su relación con nuestro trabajo es el uso del mismo corpus inglés; su diferencia, el enfoque multilingüe y profundo, que recuerda que un modelo entrenado en un idioma no se transfiere automáticamente a otro.

Maqsood et al. (2023), *An intelligent framework based on deep learning for SMS and e-mail spam detection* (Pakistán, Arabia Saudita y Reino Unido). Su objetivo fue comparar SVM, Naive Bayes multinomial, bosque aleatorio y una CNN para spam en SMS y correo. Usaron tokenización, minúsculas, *stemming*, bolsa de palabras y TF-IDF, con particiones de aproximadamente 67.7 % para entrenamiento en SMS. La muestra SMS se describe como 5000 mensajes de Kaggle (750 spam y 4250 legítimos). La SVM obtuvo accuracy de 99.6 % en SMS (secciones 4.4 y 5); las demás métricas figuran en la Tabla 4 del artículo. El artículo presenta inconsistencias en sus proporciones y en la descripción de la partición, lo que limita su uso como referencia cuantitativa.

Abdel-Jaber (2025), *Detecting spam and ham SMS messages using natural language processing and machine learning algorithms* (financiado por la Arab Open University; la afiliación no figura en la copia consultada). Su objetivo fue proponer un modelo de procesamiento de lenguaje natural y aprendizaje automático para separar SMS spam y legítimos. Aplicó limpieza, eliminación de palabras vacías, *stemming* o lematización, CountVectorizer, partición estratificada aleatoria repetida diez veces y sobremuestreo aleatorio en los experimentos balanceados, con KNN, árboles, bosque aleatorio, *gradient boosting*, MLP y SVM. La muestra fue la versión de Kaggle del corpus, con 5572 mensajes (747 spam y 4825 legítimos). Con datos desbalanceados y *stemming*, la SVM obtuvo accuracy 0.9740, precision 0.9709, recall 1.000 y F1 0.9852, y el MLP, accuracy 0.9865 y F1 0.9923 (Tablas 7–10). El artículo codifica spam = 0 y ham = 1, y la relación

entre su accuracy y su precision indica que estas métricas se calcularon con los mensajes legítimos como clase positiva; por ello no son comparables con nuestro F1 de spam.

Şahin et al. (2026), *Revisiting SMS spam detection: The impact of feature representation on classical machine learning models* (Türkiye). Su objetivo fue evaluar cómo la representación del texto afecta a clasificadores clásicos, con atención explícita a los falsos positivos. Compararon siete representaciones (conteos, bolsa binaria, TF-IDF, n-gramas de palabras, n-gramas de caracteres, TF-IDF ampliado e híbrida) con Naive Bayes, SVM lineal, bosque aleatorio, KNN y regresión logística, mediante validación cruzada estratificada de diez pliegues, ajustando la vectorización en cada pliegue, sin balanceo y con spam como clase positiva. La muestra fueron los 5574 mensajes del corpus de UCI. La mejor combinación, trigramas de caracteres con regresión logística, alcanzó accuracy 98.55 % (±0.47), precision 98.55 %, recall 90.50 %, F1 94.32 % y AUC 0.9893 (Tabla 5), con 10 falsos positivos y 71 falsos negativos (Figura 4). Es el antecedente metodológicamente más cercano; sin embargo, afirma que el corpus «no contiene mensajes duplicados» (sección 3.1), lo que contradice nuestro diagnóstico del mismo archivo.

Bani Younes y Ababneh (2026), *Machine learning based spam detection in digital communication systems: A comparative analysis* (Jordania). Su objetivo fue comparar modelos básicos de aprendizaje automático para spam en correo, SMS y redes sociales, y medir su generalización a conjuntos no vistos. Usaron TF-IDF limitado a 500 términos, eliminaron duplicados y aplicaron SMOTE sólo al entrenamiento, con Naive Bayes, SVM, bosque aleatorio, KNN, árboles y regresión logística. Para SMS emplearon 5100 mensajes del corpus de UCI y un conjunto de Kaggle como prueba externa. En el mismo dominio informaron accuracy de aproximadamente 85–96 % y precision y recall de 75–85 %; en la prueba entre dominios, la accuracy bajó a 75–83 % y la precision y el recall a 45–65 % (Figura 6 y texto). Aporta evidencia directa de que el desempeño medido en un corpus no se mantiene en otro.

Antecedentes nacionales

Calero Sinche et al. (2024), *Application of classification algorithms for smishing detection on mobile devices: Literature review* (Universidad Tecnológica del Perú, Lima). Su objetivo fue identificar las técnicas y métodos de clasificación utilizados para detectar *smishing* en dispositivos móviles. Se trata de una revisión de literatura con búsqueda en Scopus y ARDI; la población fueron estudios publicados, de los que se recuperaron 184 (155 de Scopus y 29 de ARDI) y se incluyeron 36. Los algoritmos más utilizados fueron bosque aleatorio, árboles de decisión y redes neuronales; las métricas de desempeño propias no aplican por tratarse de una revisión. Es complementario: el *smishing* es una forma de spam fraudulento con enlaces o solicitudes de datos, pero no equivale a la tarea general de separar spam y mensajes legítimos que aborda nuestro proyecto.

Reyes et al. (2026), *Predictive model of phishing attacks using machine learning for fintech companies in Peru* (Universidad Peruana de Ciencias Aplicadas, Lima; actas de congreso). Su objetivo fue clasificar correos de *phishing* para empresas *fintech*, combinando XGBoost con SMOTE. La muestra no se describe en la página consultada y el texto completo no fue accesible, por lo que sus métricas quedan como no verificadas. Se incluye como referencia nacional sobre detección de fraude con aprendizaje automático, aunque el canal (correo) y la tarea (*phishing*) difieren de la nuestra. No se localizó ningún artículo de revista peruano de los últimos cinco años sobre clasificación de spam SMS.

Antecedentes locales

Lulli Caceres y Ortiz Gutierrez (2024), El artículo 8 de la Ley de Delitos Informáticos en el tratamiento de los casos de fraude informático en la Segunda Fiscalía Penal Corporativa de Trujillo - 2023 (Universidad Privada del Norte, Trujillo), evaluaron la aplicación de esa norma durante 2023. Su estudio cualitativo, básico y no experimental empleó análisis documental y entrevistas, con una muestra intencional de 11 carpetas fiscales archivadas y tres fiscales especializados; el tamaño total de la población no se reporta (pp. 25–29). Identificaron dificultades de calificación jurídica, individualización de responsables y tratamiento de evidencia digital; la Tabla 14 (p. 51) documenta un caso iniciado por un SMS con una oferta de préstamo bancario. Es un antecedente contextual jurídico local: no evalúa un clasificador de spam ni estima la prevalencia del problema. La búsqueda adicional en RENATI y en los repositorios de UNT, UPAO, UCV y

UPN, con restricciones de acceso documentadas en el Anexo A, no permitió verificar un experimento local reciente de clasificación de spam SMS; ese antecedente metodológico específico permanece pendiente. La tesis de Castañeda Chávez (2013, UNT) continúa excluida de la ventana temporal.

La Tabla 1 resume los antecedentes. Las cifras proceden de corpus, versiones del corpus, idiomas y protocolos distintos, por lo que no se usan para afirmar la superioridad de un método; su contraste con nuestros resultados se discute en la sección 5.1.

Tabla 1

Síntesis de los antecedentes revisados

Estudio	Ámbito	Muestra	Método y protocolo	Resultado reportado (fuente)
Internacionales				
Ghourabi y Alohaly (2023)	Arabia Saudita, Túnez	5574 SMS (UCI), inglés	GPT-3 + conjunto ponderado; partición 80/20	Acc 0.9991; P 1.000; R 0.9932; F1 0.9966 (Tabla 3)
Salman et al. (2024)	Australia	67 018 SMS; 39.1 % spam (Tabla 4)	Modelos superficiales y profundos; 80/20 y conjunto adversarial separado	TCSVM + Word2Vec: Acc 97.8 %; F1 99 % (Tabla 12). En conjunto adversarial: F1 91 % sin ataque y 21 % con espacios (Tabla 14)
Altunay y Albayrak (2024)	Turquía	10 325 SMS, turco e inglés	CNN + GRU	Acc 0.9908; F1 0.9922, datos combinados (sección 5)
Maqsood et al. (2023)	Pakistán, Arabia Saudita, Reino Unido	5000 SMS (Kaggle)	SVM, NB, RF, CNN; ~67.7/32.3	SVM: accuracy 99.6 % (secciones 4.4 y 5)
Abdel-Jaber (2025)	No visible (financ. Arab Open University)	5572 SMS (Kaggle)	CountVectorizer; 10 particiones; sobremuestreo	SVM: Acc 0.9740; F1 0.9852 con ham como positiva (Tablas 7–10)
Şahin et al. (2026)	Türkiye	5574 SMS (UCI)	7 representaciones × 5 clasificadores; CV 10 pliegues	Trigramas de caracteres + RL: P 98.55 %; R 90.50 %; F1 94.32 % (Tabla 5)
Bani Younes y Ababneh (2026)	Jordania	5100 SMS (UCI) + prueba externa	TF-IDF 500 términos; SMOTE en entrenamiento	Entre dominios: accuracy 75–83 %; P y R 45–65 % (Figura 6)
Nacionales				
Calero Sinche et al. (2024)	Perú (Lima)	36 estudios de 184	Revisión de literatura (Scopus, ARDI)	Algoritmos más usados: RF, árboles, redes neuronales; métricas no aplica
Reyes et al. (2026)	Perú (Lima)	Correos; no reportada	XGBoost + SMOTE (congreso)	No verificado (texto completo no accesible)
Locales				
Lulli Caceres y Ortiz Gutierrez (2024)	Perú (Trujillo)	11 carpetas fiscales y 3 fiscales entrevistados	Cualitativo, básico, no experimental; análisis documental y entrevistas	Dificultades de investigación fiscal; caso iniciado por SMS (Tabla 14, p. 51). Contexto jurídico local, sin métricas ML

Nota. Acc = accuracy; P = precision; R = recall; RL = regresión logística; RF = bosque aleatorio. Salvo indicación, P, R y F1 se refieren a la clase spam según lo informado por cada artículo.

Elaboración propia a partir de los artículos citados.

1.2 Planteamiento del problema

Los afectados directos son los usuarios de telefonía móvil que reciben mensajes no solicitados, en especial quienes tienen menos experiencia con el fraude digital, porque un mensaje con un enlace o una solicitud de

datos puede derivar en pérdidas económicas o robo de credenciales. También se ven afectados los emisores legítimos —bancos, comercios, centros de salud— cuyos mensajes pueden confundirse con spam, y las organizaciones que deben filtrar mensajes sin bloquear comunicaciones válidas. Entre las causas destacan el bajo costo del envío masivo, la facilidad para suplantar remitentes, la ingeniería social y la adaptación constante del contenido para eludir los filtros (Salman et al., 2024). Las consecuencias incluyen pérdidas económicas (FTC, 2025), denuncias crecientes por fraude informático (Diario Oficial El Peruano, 2024), pérdida de confianza en el canal y, del lado del filtrado, el bloqueo de mensajes legítimos.

La necesidad técnica es doble. Primero, se requiere un clasificador que separe spam y mensajes legítimos con pocos falsos positivos, porque ocultar un mensaje legítimo tiene un costo alto. El aprendizaje supervisado es pertinente porque existe un corpus etiquetado público y porque el texto contiene señales discriminantes: en nuestro diagnóstico, el 78.3 % de los mensajes spam contiene una secuencia de cinco o más dígitos, frente a 0.06 % de los legítimos, y el 34.4 % menciona un símbolo monetario, frente a 0.4 %. Segundo, todo clasificador comete errores, y una parte de ellos se concentra en mensajes ambiguos. Si el modelo produce probabilidades calibradas, puede abstenerse cuando su confianza es baja y derivar esos mensajes a una persona; esto reduce el error de las decisiones automáticas a costa de trabajo humano, y ese compromiso debe medirse en lugar de suponerse. De ahí la pregunta central:

¿Cómo cambia el equilibrio entre los errores de clasificación y la proporción de mensajes resueltos automáticamente al incorporar revisión humana de predicciones inciertas en un detector de spam?

1.3 Objetivos

Objetivo general (OG). Aplicar y evaluar algoritmos de aprendizaje supervisado para detectar spam en mensajes SMS, incorporando un mecanismo de revisión humana de predicciones inciertas.

Objetivos específicos.

- **OE1:** Definir la situación actual del problema mediante la revisión de evidencia, el diagnóstico del dataset y el establecimiento de una referencia de evaluación. *Verificable* mediante la matriz bibliográfica, el archivo de diagnóstico y la métrica de la línea base.
- **OE2:** Diseñar e implementar la solución de IA/ML, incluyendo la clasificación y el mecanismo de revisión. *Verificable* mediante el pipeline persistido, el código ejecutado, las pruebas unitarias y el recorrido funcional de la aplicación.
- **OE3:** Analizar cuantitativamente los resultados y el compromiso entre errores y cobertura automática. *Verificable* mediante las métricas de prueba, la tabla de umbrales y el análisis de errores trazables a archivos de salida.

1.4 Alcance, restricciones y criterios de éxito

Funcionalidades. El sistema recibe un único mensaje escrito en un formulario web, estima la probabilidad de spam y devuelve una de tres decisiones: legítimo, spam o pendiente de revisión. Los mensajes pendientes aparecen en una bandeja donde un revisor confirma la etiqueta; la confirmación se almacena para análisis de errores y puede exportarse en CSV. Quedan fuera del alcance el análisis de enlaces o remitentes, la detección específica de *phishing*, los canales WhatsApp y correo electrónico, el reentrenamiento con las revisiones (aprendizaje activo), la autenticación de revisores y el despliegue en producción.

Idioma y datos. El modelo se entrena y evalúa sólo con mensajes en inglés del SMS Spam Collection v.1 (Almeida y Hidalgo, 2011). No se usan mensajes reales de usuarios ni datos peruanos. **Recursos y restricciones.** El proyecto dispone de cuatro estudiantes, ocho semanas, software libre y computadoras personales cuyas características deben registrarse **[POR COMPLETAR: hardware del grupo]**; no se usan servicios de pago ni modelos de lenguaje de gran tamaño en el clasificador.

Hipótesis de trabajo. H1: al menos uno de los modelos (regresión logística o SVM lineal con TF-IDF) supera en validación a la referencia de clase mayoritaria en F1 de spam. H2: para el modelo seleccionado existe un umbral $\tau \in \{0.60, 0.70, 0.80, 0.90, 0.95\}$ que reduce el error entre los mensajes aceptados respecto de $\tau = 0.50$ con una tasa de revisión de 15 % o menos en validación. H3: el modelo congelado alcanza en prueba un F1 de spam de al menos 0.90 y una precision de spam de al menos 0.95.

Los criterios de la Tabla 2 se fijaron en las instrucciones del proyecto y en el código del notebook antes de ejecutarlo; no existe un registro externo con sello de tiempo previo a la ejecución, limitación que se declara. Las metas de F1 y precision son propuestas del proyecto, no exigencias numéricas del docente, y no se modificaron después de observar los resultados. Su cumplimiento sólo indica desempeño en la partición de prueba del mismo corpus; no autoriza a generalizar a otros idiomas, operadores, periodos ni a usuarios reales.

Tabla 2

Criterios de éxito y procedimiento de evaluación

Criterio	Umbral	Datos	Procedimiento
F1 de spam (meta del proyecto)	≥ 0.90	Prueba	Una sola evaluación del modelo congelado; IC 95 % por *bootstrap* de 2000 remuestreos
Precision de spam (meta del proyecto)	≥ 0.95	Prueba	Igual que el anterior; prioriza evitar el bloqueo de mensajes legítimos
Selección del modelo	Mayor F1 de spam; si $\|\Delta F1\| < 0.005$, menor Brier	Validación	Comparación con umbral 0.5, sin abstención
Selección de τ	Error entre aceptados ≤ 1 % y revisión ≤ 15 %	Validación	Entre los τ factibles, el de mayor cobertura; empate: menor τ
Funcionamiento del prototipo	Clasifica, se abstiene, lista y guarda revisiones	Recorrido	Recorrido automatizado en navegador y pruebas unitarias

Elaboración propia.

II. MARCO TEÓRICO

2.1 Spam y clasificación supervisada de textos

Se entiende por spam SMS el mensaje de texto no solicitado y enviado de forma masiva, con fines comerciales o fraudulentos; el *smishing* es el subconjunto que busca obtener datos o dinero mediante enlaces o solicitudes engañosas (Calero Sinche et al., 2024). En términos de aprendizaje automático, el problema se formula como clasificación binaria supervisada: a partir de pares (x_i, y_i), donde x_i es el texto de un mensaje e y_i su etiqueta, se aprende una función que asigna a cada mensaje nuevo una probabilidad o una clase. El aprendizaje es supervisado porque las etiquetas fueron asignadas por los creadores del corpus (Almeida et al., 2011).

2.2 Características de entrada y etiqueta observada

La entrada es el texto bruto del mensaje, que se transforma en un vector numérico disperso (sección 2.3). La etiqueta observada toma dos valores: $y = 0$ para mensajes legítimos (*ham*) e $y = 1$ para spam. «Pendiente de revisión» no es una tercera clase: es una decisión operativa por la cual el sistema se abstiene de etiquetar. En el diseño experimental, el algoritmo, sus hiperparámetros y el umbral τ son factores manipulados; las métricas de desempeño son las variables de respuesta; y la versión del corpus, la partición y la semilla se mantienen controladas. El experimento compara configuraciones bajo condiciones idénticas, pero no permite atribuir relaciones causales entre palabras concretas y el carácter fraudulento de un mensaje.

2.3 TF-IDF

La representación TF-IDF pondera cada término t de un documento d por su frecuencia en el mensaje y por su rareza en la colección (Salton y Buckley, 1988). En la implementación utilizada, con IDF suavizado y normalización euclídea, el peso y su normalización son:

$$w(t,d) = tf(t,d) \cdot [ln\frac{1+n}{1+df(t)} + 1] \quad (1)$$

$$x_d = \frac{v_d}{\|v_d\|_2} \quad (2)$$

donde tf(t, d) es el número de apariciones de t en d, n el número de mensajes de entrenamiento, df(t) el número de mensajes que contienen t y v_d el vector de pesos del mensaje antes de normalizarlo. Los términos pueden ser unigramas o pares de palabras consecutivas (bigramas). El vocabulario, n y df(t) se aprenden exclusivamente con los datos de ajuste; aplicarlos sobre validación o prueba no modifica esos valores.

2.4 Regresión logística

La regresión logística modela la probabilidad de spam mediante la función sigmoide de una combinación lineal de las características (ecuación (3)). Sus parámetros se estiman minimizando la pérdida logística regularizada de la ecuación (4), donde $\tilde{y}_i \in \{-1, +1\}$ recodifica la etiqueta y $C > 0$ es el inverso de la fuerza de regularización L2: valores pequeños de C penalizan más los pesos y favorecen modelos más simples. La configuración utilizada resuelve el problema con el optimizador L-BFGS, de tipo cuasi-Newton (Pedregosa et al., 2011).

$$p(x) = P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1+e^{-(w^\top x+b)}} \quad (3)$$

$$min_{w,b}\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} ln(1 + e^{-\tilde{y}_i(w^\top x_i+b)}) \quad (4)$$

2.5 Máquina de vectores de soporte lineal

La SVM busca un hiperplano que separe las clases con el mayor margen posible (Cortes y Vapnik, 1995). La variante lineal utilizada minimiza la pérdida bisagra al cuadrado con regularización L2 (ecuación (5)) y se resuelve mediante descenso por coordenadas en la biblioteca LIBLINEAR (Fan et al., 2008). Su salida es una puntuación $f(x) = w^\top x + b$ cuyo signo determina la clase; esa puntuación **no es una probabilidad**, por lo que, para usarla como medida de confianza, debe calibrarse (sección 2.6).

$$min_{w,b}\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} max(0, 1 - \tilde{y}_i(w^\top x_i + b))^2 \quad (5)$$

2.6 Calibración de probabilidades

Un clasificador está calibrado cuando, entre los mensajes a los que asigna probabilidad p, la proporción real de spam es aproximadamente p (Silva Filho et al., 2023). La calibración sigmoide de Platt (2000) transforma la puntuación de la SVM mediante la ecuación (6), donde A y B se estiman por máxima verosimilitud con datos que **no** se usaron para ajustar la SVM. En este trabajo se aplica validación cruzada interna: cada SVM se ajusta en cuatro pliegues del entrenamiento y su sigmoide en el quinto, y la probabilidad final promedia los cinco modelos calibrados. La calidad de la calibración se evalúa con la puntuación de Brier (ecuación (7)) y con el error de calibración esperado (ECE) sobre la confianza (ecuación (8)), calculado con diez intervalos de igual ancho en [0.5, 1] (Silva Filho et al., 2023, sección 4.4). Los diagramas de fiabilidad complementan esta evaluación (Niculescu-Mizil y Caruana, 2005).

$$p(x) = \frac{1}{1+exp(A\cdot f(x)+B)} \quad (6)$$

$$BS = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2 \quad (7)$$

$$ECE = \sum_{b=1}^{B}\frac{n_b}{n}|acc_b - conf_b| \quad (8)$$

2.7 Clasificación selectiva y abstención

La clasificación con opción de rechazo permite que el modelo no decida cuando la incertidumbre es alta (Chow, 1970). Hendrickx et al. (2024) distinguen el rechazo por ambigüedad, cuando el ejemplo está cerca de la frontera entre clases, del rechazo por novedad, cuando el ejemplo difiere de los datos de entrenamiento;

este trabajo implementa sólo el primero. La confianza se define como la mayor de las probabilidades de clase (ecuación (9)) y la regla de decisión, como en la ecuación (10). En caso de empate, p(x) = 0.5, se asigna la etiqueta legítimo; como en ese caso c(x) = 0.5 < τ para todo $\tau > 0.5$, el mensaje termina en revisión. Con $\tau = 0.50$ no hay abstención posible, porque c(x) ≥ 0.5 siempre.

$$c(x) = max\{p(x), 1 - p(x)\} \tag{9}$$

$$\delta_\tau(x) = \{revisión\ si\ c(x) < \tau;\ \ spam\ si\ c(x) \geq \tau\ \wedge\ p(x) > 0.5;\ \ legítimo\ en\ otro\ caso\} \tag{10}$$

El compromiso entre errores y trabajo humano se mide con la cobertura automática $\varphi(\tau)$, proporción de mensajes que el sistema decide (ecuación (11)), y con el error entre aceptados o riesgo selectivo $R(\tau)$ (ecuación (12)). La tasa de revisión es $1 - \varphi(\tau)$. Una disminución de $R(\tau)$ no implica una mejora global: los mensajes en revisión siguen sin resolverse hasta que una persona los atiende, y la calidad final depende de esa revisión.

$$\varphi(\tau) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[c(x_i) \geq \tau] \tag{11}$$

$$R(\tau) = \frac{\sum_{i=1}^{n} \mathbb{1}[\hat{y}_i \neq y_i] \cdot \mathbb{1}[c(x_i) \geq \tau]}{\sum_{i=1}^{n} \mathbb{1}[c(x_i) \geq \tau]} \tag{12}$$

Para la clase spam se informan precision, recall y F1 (ecuación (13)), donde VP, FP y FN son verdaderos positivos, falsos positivos y falsos negativos con spam como clase positiva. La *accuracy* se reporta sólo como medida complementaria, porque con 13.4 % de spam un clasificador que nunca detecta spam alcanza alrededor de 87 %.

$$P = \frac{VP}{VP+FP},\ \ R = \frac{VP}{VP+FN},\ \ F1 = \frac{2PR}{P+R} \tag{13}$$

2.8 Revisión humana

El aprendizaje automático con intervención humana incorpora a las personas en el ciclo de decisión o de aprendizaje: como etiquetadoras, como supervisoras de casos difíciles o como fuente de correcciones para reentrenar (Mosqueira-Rey et al., 2023). En este proyecto la persona interviene **sólo en la decisión** de los mensajes en los que el modelo se abstiene. Guardar sus correcciones prepara datos para mejoras futuras, pero no constituye aprendizaje activo ni demuestra que el modelo mejore; ello exigiría reentrenar y evaluar de nuevo con datos independientes. Además, el revisor puede equivocarse o ser inconsistente, por lo que el supuesto de un revisor perfecto sólo se usa como cota optimista y se identifica como simulación.

2.9 Desbalance, generalización, sobreajuste y fuga de información

El corpus está desbalanceado (13.4 % de spam), lo que justifica informar métricas de la clase minoritaria. No se aplicó sobremuestreo, submuestreo ni ponderación de clases: los modelos lineales regularizados con TF-IDF ya alcanzaban precision alta y el objetivo prioritario era no bloquear mensajes legítimos, mientras que el reequilibrio tiende a desplazar la frontera hacia más falsos positivos; cualquier técnica de balanceo futura deberá aplicarse sólo al entrenamiento. Tampoco se aplicó escalado adicional, porque cada vector TF-IDF ya tiene norma euclídea unitaria, ni selección de variables, porque la regularización L2 controla la dimensionalidad. La **generalización** es el desempeño en datos no usados para ajustar; el **sobreajuste** se diagnostica comparando el desempeño de ajuste con el de los pliegues excluidos en curvas de aprendizaje. La **fuga de información** ocurre cuando información de evaluación influye en el entrenamiento o en las decisiones de diseño (Kaufman et al., 2012). Kapoor y Narayanan (2023) documentan, entre sus formas frecuentes, el preprocesamiento ajustado con todos los datos, los duplicados entre entrenamiento y prueba y la reutilización de la prueba para seleccionar modelos; el protocolo de la sección 3.4 se diseñó para evitar las tres.

2.10 Herramientas y frameworks

La Tabla 3 justifica las herramientas elegidas. El criterio fue usar componentes libres, documentados y adecuados para un proyecto de ocho semanas, evitando dependencias que el grupo no pueda mantener.

Tabla 3

Herramientas utilizadas: función, ventajas y limitaciones

Herramienta	Función en el proyecto	Ventajas	Limitaciones
Python	Lenguaje de todo el proyecto	Ecosistema amplio para datos y web	Requiere fijar versiones para reproducir
pandas (The pandas development team, s. f.)	Lectura del corpus y tablas de resultados	Manejo tabular simple; exporta CSV	El comillado por defecto altera este archivo (sección 3.5)
scikit-learn	TF-IDF, RL, SVM, CV agrupada, calibración, métricas	*Pipelines* que impiden ajustar el preprocesamiento fuera del pliegue (Pedregosa et al., 2011)	Sin aprendizaje incremental en estos modelos; los artefactos dependen de la versión
Joblib (Joblib developers, s. f.)	Persistencia del pipeline y de τ	Serialización eficiente de objetos NumPy	Formato basado en *pickle*: no cargar archivos de origen desconocido
Flask (Pallets, s. f.)	Aplicación web: formulario, bandeja y revisiones	Microframework ligero, rutas explícitas	Servidor de desarrollo no apto para producción
SQLite (SQLite Consortium, s. f.)	Almacenamiento de mensajes y revisiones	Sin servidor; un único archivo	Concurrencia de escritura limitada; requiere cifrado externo

Elaboración propia.

2.11 Pseudocódigo del entrenamiento, la evaluación y la decisión de revisión

El Listado 1 resume el procedimiento de entrenamiento y selección, y el Listado 2, la regla de decisión que usan tanto el experimento como la aplicación. Ambos corresponden a la configuración realmente implementada (secciones 3.4 y 3.6).

Listado 1

Pseudocódigo del entrenamiento, la selección y la evaluación

```
Entrada: D = {(x_i, y_i)}, semilla 42, rejilla G = C ∈ {0.1, 1, 10} × n-gramas ∈ {(1,1), (1,2)}
 1  (Tr, Va, Te) ← PartirEstratificadoAgrupado(D, grupo = texto normalizado, 60/20/20)
 2  K ← CincoPlieguesEstratificadosAgrupados(Tr)
 3  para cada algoritmo a ∈ {RL, SVM}:
 4     θ*_a ← argmax_{θ ∈ G} media_k F1_spam(Ajustar(TFIDF + a_θ, Tr \ K_k), K_k)
 5     si a = RL:  M_a ← Ajustar(TFIDF + RL_θ*, Tr)
 6     si a = SVM: M_a ← promedio_k Platt(Ajustar(TFIDF + SVM_θ*, Tr \ K_k), K_k)
 7  a* ← argmax_a F1_spam(M_a, Va)        ▷ si |ΔF1| < 0.005: menor Brier en Va
 8  τ* ← ElegirUmbral(M_a*, Va, error_aceptados ≤ 0.01, revisión ≤ 0.15)
 9  Congelar(M_a*, τ*); Guardar(joblib, sha256)
10  Informar(MétricasBinarias y Selectivas(M_a*, τ*, Te))   ▷ única evaluación en prueba
```

Elaboración propia; implementado en el notebook `01_experimento_spam_sms.ipynb`.

Listado 2

Pseudocódigo de la decisión con revisión humana

```
función Decidir(x, M, τ):
   p ← M.probabilidad_spam(x)       ▷ probabilidad calibrada
   c ← máx(p, 1 − p)                ▷ confianza
   si c < τ: devolver "pendiente_revision"   ▷ abstención; no es una clase
   si p > 0.5: devolver "spam"
   devolver "legitimo"              ▷ incluye el empate p = 0.5 cuando τ = 0.50

función Revisar(id_mensaje, etiqueta ∈ {0, 1}, revisor):
   guardar (id_mensaje, etiqueta, revisor, fecha) en SQLite   ▷ no reentrena el modelo
```

Elaboración propia; implementado en `src/spam_sms/decision.py` y `app/app.py`.

III. METODOLOGÍA

3.1 Metodología de la investigación

Enfoque y tipo. La investigación es cuantitativa y aplicada: construye un artefacto de software y mide su desempeño con métricas definidas de antemano. **Diseño.** Es un diseño experimental computacional comparativo con datos retenidos: se manipulan el algoritmo, sus hiperparámetros y el umbral τ, y se comparan bajo las mismas particiones y semilla. No es un experimento con personas: no hay participantes humanos, encuestas ni pruebas con usuarios, y el «revisor» de la aplicación se representa en una demostración funcional, no en un estudio de usabilidad.

Población, muestra y unidad de análisis. La unidad de observación es un mensaje SMS; el número de mensajes no equivale al número de personas, porque una persona pudo aportar varios mensajes y el corpus no identifica remitentes. La población accesible son los 5574 mensajes en inglés del SMS Spam Collection v.1 y se trabajó con el censo de esa población, dividida en entrenamiento, validación y prueba (sección 3.4). **Inclusión:** toda línea del archivo oficial con etiqueta *ham* o *spam* y texto no vacío. **Exclusión:** ninguna línea cumplió criterios de exclusión (0 mensajes vacíos y 0 textos con etiquetas contradictorias); los duplicados se conservaron, pero se agruparon para que no crucen particiones.

Técnicas e instrumentos. Se emplearon el análisis documental, con una matriz bibliográfica (Anexo A); la experimentación computacional, con un notebook de Jupyter y módulos de Python versionados en la carpeta del proyecto; la prueba de software, con pruebas unitarias en pytest y un recorrido automatizado de la aplicación en un navegador; y el registro de evidencias, mediante archivos JSON y CSV generados por el propio código, con huellas SHA-256 del corpus y del modelo.

3.2 Materiales y métodos

La Tabla 4 lista el software y las versiones usadas en la ejecución documentada. El hardware comprobado corresponde al entorno de ejecución en la nube donde se corrieron el notebook y la aplicación: Linux x86_64 con dos núcleos lógicos (Intel Xeon a 2.10 GHz) y unos 7 GB de memoria. Las características de las computadoras del grupo no se han registrado **[POR COMPLETAR: procesador, memoria y sistema operativo de cada integrante]**, y el notebook deberá volver a ejecutarse en ellas antes de la sustentación. El trabajo se organizó según CRISP-DM (Chapman et al., 2000), cuyas fases se vinculan con evidencias en la sección 3.3.

Tabla 4

Software y versiones de la ejecución documentada

Componente	Versión	Uso
Python	3.13.16	Intérprete
NumPy / pandas	2.5.3 / 3.0.5	Cálculo numérico y tablas
scikit-learn	1.9.1	TF-IDF, RL, SVM lineal, CV agrupada, calibración y métricas
Joblib	1.6.0	Persistencia del pipeline
Matplotlib / Graphviz	3.11.2 / 2.43.0	Figuras de resultados / diagramas conceptuales
Flask / Werkzeug / Jinja2	3.1.3 / 3.1.8 / 3.1.6	Aplicación web
SQLite (biblioteca)	3.45.1	Base de datos de mensajes y revisiones
nbconvert / ipykernel	7.17.2 / 7.4.0	Ejecución no interactiva del notebook
pytest / Playwright	9.1.1 / 1.56.0	Pruebas unitarias / recorrido de la aplicación y capturas

Elaboración propia a partir de `results/entorno_ejecucion.json` y de los paquetes instalados.

3.3 Proceso de desarrollo

La Tabla 5 relaciona cada fase de CRISP-DM con las actividades previstas en las ocho semanas del proyecto y con la evidencia disponible. El calendario es una planificación; el grupo debe registrar las fechas reales de avance **[POR COMPLETAR: fechas por semana]**.

Tabla 5

Fases CRISP-DM, actividades, evidencias y estado

Semana	Fase CRISP-DM	Actividades	Evidencia	Estado
1	Comprensión del negocio	Problema, alcance, objetivos, búsqueda bibliográfica	Secciones I y 1.4; Anexo A	Hecho; antecedente contextual local incorporado; clasificador local pendiente
2	Comprensión de los datos	Verificación del ZIP oficial, clases, duplicados, contradicciones	diagnostico_dataset.json	Hecho
3	Preparación de los datos	Partición agrupada 60/20/20, línea base	particiones.csv; linea_base_validacion.json	Hecho
4	Modelado	Rejilla y CV agrupada para RL y SVM; curvas de aprendizaje	cv_rejilla.csv; curva_aprendizaje_*.csv	Hecho
5	Modelado y evaluación	Calibración, comparación en validación, elección de τ	calibracion_validacion.csv; umbrales_validacion_*.csv	Hecho
6	Despliegue (prototipo)	Flask + SQLite: formulario, bandeja, revisión	app/; capturas; pruebas unitarias	Prototipo local; sin despliegue
7	Evaluación	Congelamiento, prueba única, análisis de errores	decisiones_congeladas.json; evaluacion_final_prueba.json	Hecho
8	Comunicación	Artículo, sustentación, revisión del código por el grupo	Este documento	En curso

Nota. Las evidencias se encuentran en la carpeta codigo/results/. El estado «Hecho» indica que existe una ejecución verificable; la autoría y la comprensión del código por cada integrante deben acreditarse aparte (sección 6.3).

Elaboración propia.

3.4 Diseño del experimento

Particiones. El corpus se dividió en entrenamiento (60 %), validación (20 %) y prueba (20 %) con estratificación por clase y agrupación por duplicados, con semilla 42. El grupo de un mensaje es su texto en minúsculas con los espacios colapsados; esta clave sólo se usa para agrupar y no altera el texto que ve el modelo. Primero, `StratifiedGroupKFold` con cinco pliegues separó la prueba (un pliegue); después, el mismo procedimiento con cuatro pliegues sobre el resto separó la validación. El código verifica que ningún grupo aparezca en dos particiones (Tabla 9).

Variables y controles. La Tabla 6 resume los factores y las variables de respuesta. Se controlaron la versión del archivo (huella SHA-256), la semilla, las particiones, los pliegues internos —los mismos cinco para la búsqueda de hiperparámetros, las curvas de aprendizaje y la calibración— y la configuración del vectorizador, idéntica para ambos algoritmos.

Tabla 6
Factores, controles y variables de respuesta

Elemento	Tipo	Valores o definición
Algoritmo	Factor	Regresión logística (L-BFGS, L2) o SVM lineal (pérdida bisagra al cuadrado, L2)
C	Factor	0.1, 1, 10
n-gramas de palabras	Factor	(1, 1) unigramas o (1, 2) unigramas y bigramas
Umbral τ	Factor	0.50, 0.60, 0.70, 0.80, 0.90, 0.95
Vectorizador	Control	TF-IDF, minúsculas, IDF suavizado, norma L2, min_df = 1, sin eliminación de palabras vacías
Partición y semilla	Control	60/20/20 estratificada y agrupada; semilla 42; mismos cinco pliegues internos
Desempeño binario	Respuesta	Precision, recall y F1 de spam; matriz de confusión; FP; FN; *accuracy* complementaria
Calibración	Respuesta	Brier, pérdida logarítmica, ECE (10 intervalos sobre la confianza)
Desempeño selectivo	Respuesta	Cobertura, tasa de revisión, error entre aceptados, métricas por clase y distribución del subconjunto aceptado

Elaboración propia.

Candidatos y selección. Se evaluaron doce configuraciones: dos algoritmos por tres valores de C por dos rangos de n-gramas. Para cada algoritmo, la configuración con mayor F1 de spam medio en la validación cruzada agrupada de cinco pliegues dentro del entrenamiento se reajustó sobre todo el entrenamiento. Como el vectorizador forma parte del *pipeline*, su vocabulario y sus pesos IDF se aprendieron de nuevo en cada pliegue.

Calibración. La regresión logística usa sus probabilidades nativas. La SVM se calibró con la sigmoide de Platt mediante `CalibratedClassifierCV` con los cinco pliegues agrupados del entrenamiento: cada SVM se ajustó en cuatro pliegues, su sigmoide en el pliegue excluido, y la predicción promedia los cinco modelos. La validación no intervino en ningún ajuste; se usó para medir la calibración (Brier, pérdida logarítmica y ECE) y para decidir.

Decisiones en validación. (1) Modelo: mayor F1 de spam con umbral 0.5; si la diferencia era menor que 0.005, se elegiría el de menor Brier y, de persistir el empate, la regresión logística. (2) Umbral: entre los τ con error entre aceptados ≤ 1 % y tasa de revisión ≤ 15 %, el de mayor cobertura (empate: menor τ); si ninguno cumplía, se elegiría el de menor error con revisión ≤ 15 %. Ambas reglas se escribieron en el código antes de ejecutarlo.

Congelamiento y evaluación final. Tras las decisiones, el notebook guarda el *pipeline* entrenado sólo con el entrenamiento y τ en un archivo Joblib, junto con `decisiones_congeladas.json`, que registra el modelo, los hiperparámetros, τ, las metas y la huella SHA-256 del artefacto. Sólo entonces se carga el artefacto desde disco y se evalúa la prueba. Se calcularon intervalos de confianza al 95 % por percentiles de 2000 remuestreos *bootstrap* de los mensajes de prueba (semilla 42). Como análisis de sensibilidad, se repitieron las métricas sobre la prueba sin duplicados internos. La tabla de todos los τ en prueba es descriptiva y no se usó para elegir. El sistema con «revisión ideal» —el revisor asigna la etiqueta del corpus— es una simulación y una cota optimista, no una prueba con usuarios.

Reproducibilidad. La Figura 1 resume el protocolo. El notebook se ejecutó de principio a fin tres veces durante la sesión de trabajo: la primera produjo los resultados y las otras dos, motivadas sólo por correcciones de figuras, reprodujeron métricas de prueba idénticas (comparación automática de los archivos JSON) y el mismo artefacto (SHA-256 `de87978…5482705`). Ninguna decisión se modificó después de la primera evaluación de la prueba.

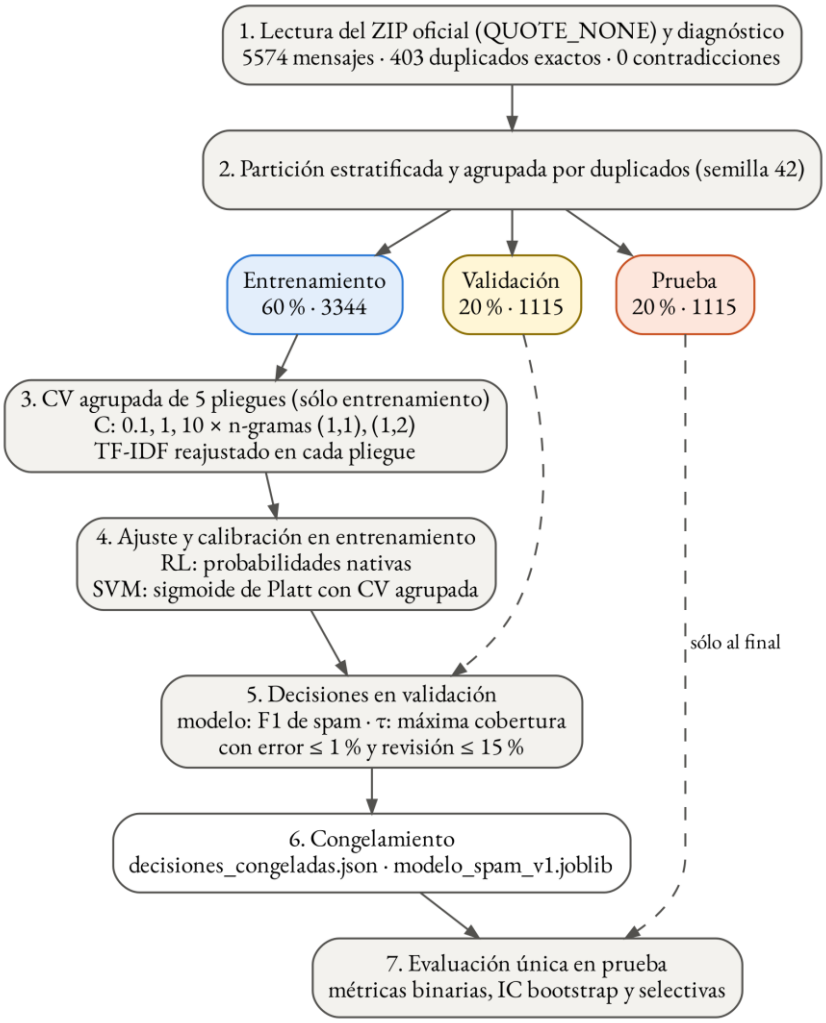


Figura 1

Protocolo experimental implementado

Elaboración propia. Diagrama conceptual; las cifras de los nodos 1 y 2 proceden de `diagnostico_dataset.json` y `particiones_resumen.csv`.

3.5 Dataset y preparación de datos

Origen y licencia. Se usó el SMS Spam Collection publicado en el UCI Machine Learning Repository (Almeida y Hidalgo, 2011), DOI 10.24432/C5CC84, con licencia Creative Commons Atribución 4.0 Internacional según la ficha del repositorio, donado el 21 de junio de 2012. El artículo de origen del corpus (Almeida et al., 2011) se cita como fuente de los datos y no como antecedente, por su antigüedad. El ZIP oficial (203 415 bytes; SHA-256 `1587ea43…99963ce3`) fue descargado por el grupo desde la página de UCI y contiene el archivo `SMSSpamCollection`, con fecha del 15 de marzo de 2011, y un `readme` que lo identifica como «SMS Spam Collection v.1». Según ese documento, los mensajes proceden del foro británico Grumbletext (425 spam), de la tesis de Caroline Tagg (450 legítimos), del NUS SMS Corpus de Singapur (3375 legítimos) y del SMS Spam Corpus v.0.1 Big (1002 legítimos y 322 spam).

Estructura. Es un archivo de texto UTF-8 sin encabezado, con una línea por mensaje y dos campos separados por tabulador: la etiqueta y el texto bruto (Tabla 7). Una decisión de lectura resultó crítica: con el comillado por defecto de pandas, las comillas dobles internas de algunos mensajes fusionan líneas y se obtienen 5572 filas en lugar de 5574; por eso el lector usa `csv.QUOTE_NONE` y comprueba la cantidad de filas. Esta diferencia coincide con el tamaño que informan los estudios que usan la versión de Kaggle del corpus (Abdel-Jaber, 2025).

Tabla 7

Diccionario de datos del corpus y de las variables derivadas

Variable	Tipo	Valores	Descripción
label	Categórica	ham, spam	Etiqueta asignada por los creadores del corpus (campo 1 del archivo)
text	Texto	2 a 910 caracteres	Mensaje SMS original en inglés, sin modificar (campo 2)
y	Binaria	0 = ham, 1 = spam	Codificación usada por los modelos y las métricas
grupo	Texto derivado	5159 valores	Texto en minúsculas con espacios colapsados; sólo para agrupar duplicados
particion	Categórica derivada	entrenamiento, validación, prueba	Asignación de la partición agrupada (particiones.csv)
p_spam	Numérica derivada	[0, 1]	Probabilidad calibrada de spam estimada por el modelo
decision	Categórica derivada	legitimo, spam, pendiente_revision	Decisión operativa de la aplicación; no es una clase del corpus

Elaboración propia a partir del readme del corpus y del código del proyecto.

Distribución y calidad. La Tabla 8 presenta las cantidades calculadas sobre el archivo real. No hay mensajes vacíos ni textos con etiquetas contradictorias. Existen 403 filas que repiten exactamente un texto anterior (5171 textos únicos); al normalizar mayúsculas y espacios, las repeticiones suben a 415 y quedan 5159 grupos. Los duplicados no se eliminaron, porque forman parte de la distribución real del corpus, pero se agruparon para impedir que un mismo texto aparezca en entrenamiento y en evaluación. La Figura 4 (sección 4.2) muestra la distribución por clase.

Tabla 8

Diagnóstico del archivo oficial SMS Spam Collection v.1

Indicador	Legítimo (ham)	Spam	Total
Mensajes	4827	747	5574
Porcentaje	86.60 %	13.40 %	100 %
Textos únicos (sin duplicados exactos)	4518	653	5171
Filas duplicadas exactas	309	94	403
Mensajes vacíos / textos con etiqueta contradictoria	—	—	0 / 0
Longitud media (mediana), caracteres	71.5 (52)	138.7 (149)	—
Contiene enlace (http, www o .com)	0.29 %	16.20 %	—
Contiene una secuencia de 5 o más dígitos	0.06 %	78.31 %	—
Contiene símbolo monetario (£, $ o €)	0.39 %	34.40 %	—

Nota. Los indicadores textuales son descriptivos y no se usaron como variables separadas del modelo. Las filas duplicadas por clase se obtienen como mensajes menos textos únicos.

Elaboración propia a partir de diagnostico_dataset.json e indicadores_textuales.csv.

Tabla 9

Particiones estratificadas y agrupadas por duplicados (semilla 42)

Partición	Mensajes	% del total	Spam	Ham	% spam	Grupos únicos
Entrenamiento	3344	59.99	448	2896	13.40	3100
Validación	1115	20.00	149	966	13.36	1027
Prueba	1115	20.00	150	965	13.45	1032

Nota. Grupos compartidos entre particiones: entrenamiento–validación 0; entrenamiento–prueba 0; validación–prueba 0. Elaboración propia a partir de particiones_resumen.csv.

Decisiones de representación. El texto se convierte a minúsculas, pero no se eliminan números, enlaces ni símbolos: los códigos cortos, los teléfonos, los montos y los símbolos son señales frecuentes de spam (Tabla 8). El patrón de *tokens* conserva palabras alfanuméricas de uno o más caracteres —incluidas abreviaturas propias del SMS como «u», «2» o «4»— y trata como *tokens* individuales los símbolos £, $, €, !, ?, % y &; un enlace se divide en sus partes (por ejemplo, «www» y «com»). No se eliminaron palabras vacías ni se aplicó *stemming*, porque en mensajes tan cortos las palabras funcionales y las grafías no normalizadas también aportan información, y porque así se mantienen los mismos *tokens* en el entrenamiento y en la aplicación. Reemplazar números o enlaces por marcadores genéricos queda como ablación futura. Estas decisiones se fijaron antes de experimentar y no se ajustaron con validación ni con prueba.

3.6 Desarrollo de la solución IA/ML

Arquitectura. La Figura 2 muestra la arquitectura implementada. Fuera de línea, el notebook lee el ZIP oficial, ejecuta el protocolo y produce un artefacto Joblib que contiene el *pipeline* (TF-IDF y SVM lineal calibrada), τ, la versión y la codificación de etiquetas. En línea, una aplicación Flask carga ese artefacto al iniciar, recibe el texto desde un formulario, calcula la probabilidad, aplica la regla de decisión y guarda el resultado en SQLite. El reentrenamiento con revisiones aparece con línea discontinua porque **no está implementado**.

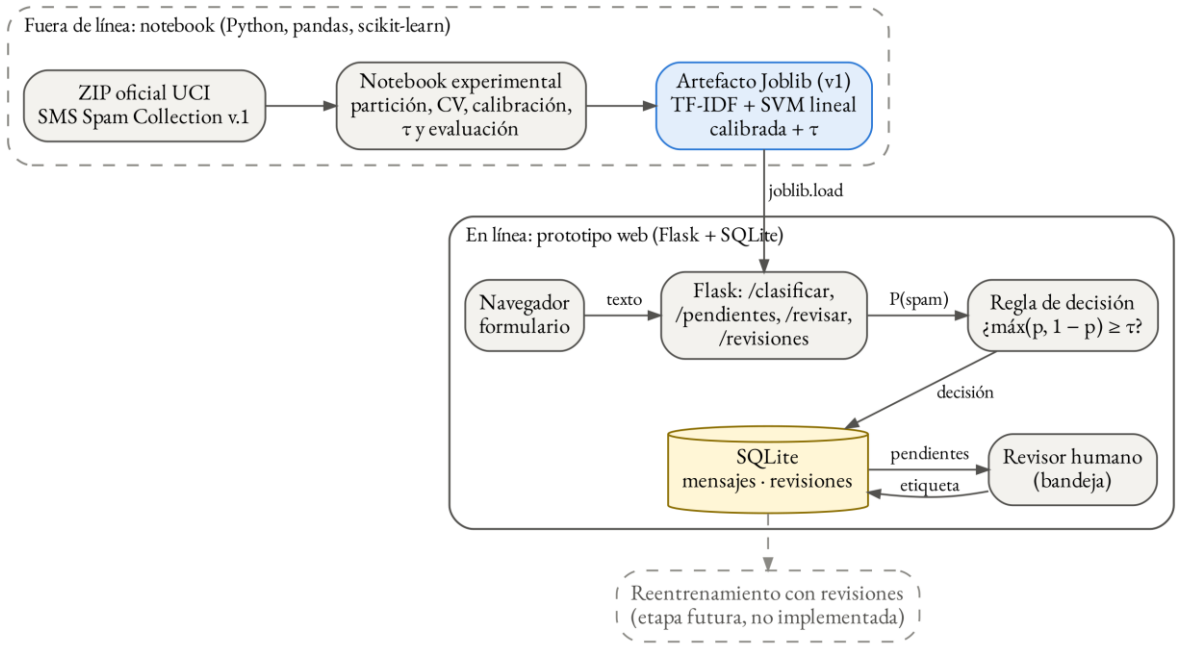


Figura 2

Arquitectura del prototipo implementado y componente futuro

Elaboración propia. Diagrama conceptual de los componentes presentes en codigo/; el reentrenamiento es una propuesta.

Flujo de decisión. La Figura 3 detalla el recorrido de un mensaje: vectorización con el vocabulario del entrenamiento, probabilidad calibrada, comparación de la confianza con τ y, si corresponde, revisión humana. La misma función `decide` se usa en el experimento y en la aplicación, por lo que las métricas selectivas describen exactamente la regla desplegada.

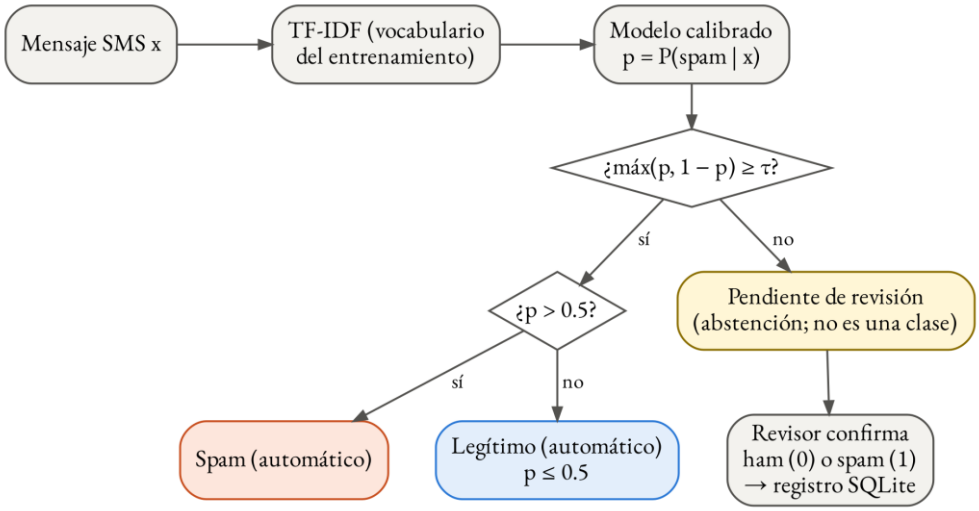


Figura 3

Flujo de clasificación con abstención y revisión humana

Elaboración propia. Diagrama conceptual de la regla implementada en `src/spam_sms/decision.py`.

Persistencia y almacenamiento. El artefacto `models/modelo_spam_v1.joblib` (4.8 MB) se identifica por su huella SHA-256. La base SQLite tiene dos tablas: `mensajes` (texto, probabilidad, confianza, etiqueta sugerida, decisión, τ, versión del modelo y fecha) y `revisiones` (mensaje, etiqueta del revisor, nombre del revisor, comentario y fecha), con restricción de una revisión por mensaje. La aplicación ofrece cuatro rutas: `/clasificar`, `/pendientes`, `/revisar/<id>` y `/revisiones`, además de la exportación `/revisiones.csv`. El Listado 3 muestra la ruta principal.

Listado 3

Ruta de clasificación de la aplicación Flask (código ejecutado)

```
@app.route("/clasificar", methods=["POST"])
def clasificar_mensaje():
    texto = (request.form.get("texto") or "").strip()
    if not texto:
        return render_template("index.html", ..., error="Escribe un mensaje antes de clasificar.")
    texto = texto[:MAX_LEN]                          # máximo 1000 caracteres
    p = float(MODELO.predict_proba([texto])[0, 1])   # probabilidad calibrada de spam
    y_hat, aceptado = decide([p], TAU)               # misma regla que el experimento
    decision = REVIEW if not aceptado[0] else (AUTO_SPAM if y_hat[0] == 1 else AUTO_HAM)
    db().execute("INSERT INTO mensajes (texto, p_spam, confianza, etiqueta_modelo, decision, "
                 "tau, version_modelo, creado_en) VALUES (?, ?, ?, ?, ?, ?, ?, ?)", (...))
    db().commit()
    return render_template("resultado.html", ...)
```

Extracto simplificado de `codigo/app/app.py`; los puntos suspensivos sustituyen argumentos omitidos. Código completo en el Anexo E.

Hiperparámetros. La Tabla 10 recoge la configuración final congelada. **Repositorio y notebook.** El código se entrega en la carpeta del proyecto, en `Proyecto del Curso/codigo/`, y todavía no tiene un repositorio remoto **[PENDIENTE: REPOSITORIO]**. El notebook está en `codigo/notebooks/01_experimento_spam_sms.ipynb` y se genera desde `build_notebook.py`. Contiene 43 celdas, 26 de ellas de código, todas ejecutadas, y sigue el orden de la Figura 1. Para completarlo en sus equipos, el grupo debe colocar el ZIP en `Proyecto del Curso/Dataset/`, instalar las dependencias de `requirements.txt` con Python 3.13 y ejecutar todas las celdas; los resultados se escriben en `codigo/results/` (instrucciones en el Anexo F).

Tabla 10
Configuración final congelada del clasificador y de la regla de revisión

Componente	Valor
Algoritmo	SVM lineal (`LinearSVC`): pérdida bisagra al cuadrado, penalización L2, `dual='auto'`, `max_iter` = 20 000
C	10 (límite superior de la rejilla evaluada)
Vectorizador	TF-IDF de unigramas y bigramas; minúsculas; patrón `(?u)\b\w+\b|[£$€!?%&]`; IDF suavizado; norma L2
Calibración	Sigmoide de Platt; `CalibratedClassifierCV` con 5 pliegues agrupados del entrenamiento; `ensemble=True`
Umbral de revisión	$\tau = 0.70$ (factible en validación: error entre aceptados 0.90 %, revisión 0.63 %)
Datos de ajuste	Sólo entrenamiento (3344 mensajes); validación y prueba no se usaron para ajustar
Artefacto	`models/modelo_spam_v1.joblib`; SHA-256 `de87978042e6...82705`

Elaboración propia a partir de `decisiones_congeladas.json` y `mejores_hiperparametros.json`.

Estado de los componentes. La Tabla 11 distingue lo desarrollado y ejecutado, lo propuesto y lo pendiente. El código fue generado con asistencia de IA y ejecutado en un entorno en la nube; que constituya avance del grupo depende de que sus integrantes lo revisen, lo comprendan y lo ejecuten en sus equipos (sección VI).

Tabla 11
Estado de los componentes de la solución

Componente	Estado	Evidencia o requisito
Desarrollados y ejecutados		
Carga verificada, diagnóstico y partición agrupada	Ejecutado	`src/spam_sms/data.py`; `diagnostico_dataset.json`; prueba unitaria
Búsqueda de hiperparámetros, calibración y selección de τ	Ejecutado	Notebook, celdas 5–10; tablas en `results/tablas/`
Evaluación final, análisis de errores y latencia	Ejecutado	`evaluacion_final_prueba.json`; `errores_prueba.csv`
Aplicación Flask: formulario, decisión, bandeja, revisión y CSV	Ejecutado (local)	`app/`; recorrido con Playwright; capturas
Pruebas unitarias del núcleo	Ejecutado	4 pruebas superadas (`results/pytest_salida.txt`)
Propuestos		
Reentrenamiento con revisiones (aprendizaje activo)	Propuesto	Requiere datos revisados suficientes y una nueva evaluación independiente
Detección de idioma y rechazo por novedad	Propuesto	Enviar a revisión los mensajes fuera del dominio del entrenamiento
Pendientes		
Repositorio remoto con historial de autoría	Pendiente	**[PENDIENTE: REPOSITORIO]**
Ejecución en los equipos del grupo	Pendiente	Repetir el notebook y comparar las huellas SHA-256
Autenticación de revisores, retención y anonimización de mensajes	Pendiente	Necesarios antes de cualquier uso con mensajes reales (sección VII)
Evaluación con revisores humanos reales	Pendiente	Medir errores de revisión y tiempos; no realizada

Elaboración propia.

IV. RESULTADOS

4.1 Justificación del proyecto

Evidencia teórica. Los mensajes SMS representados con TF-IDF producen vectores de alta dimensión y muy dispersos: con unigramas y bigramas, cada modelo del entrenamiento aprendió unos 30 000 términos, de los cuales un mensaje activa una mediana de 23. En ese escenario, los modelos lineales regularizados son eficientes de entrenar, se ejecutan en milisegundos y tienen parámetros interpretables (Fan et al., 2008; Pedregosa et al., 2011). La regresión logística estima probabilidades directamente, y la SVM lineal maximiza el margen, pero requiere calibración para producir probabilidades (Platt, 2000). Como la regla de revisión depende de la confianza, la calibración es una condición del diseño y no un detalle técnico (Silva Filho et al., 2023; Hendrickx et al., 2024).

Evidencia empírica de antecedentes. Sobre el mismo corpus, Şahin et al. (2026) informaron que una regresión logística con trigramas de caracteres alcanza F1 de spam de 94.32 % con validación cruzada de diez pliegues, y que la SVM lineal se comporta de forma comparable; Ghourabi y Alohaly (2023) obtuvieron accuracy de 0.9785 con una SVM y TF-IDF; y Maqsood et al. (2023) informaron 99.6 % de accuracy con SVM en una versión de 5000 mensajes. Estos resultados justifican elegir ambos algoritmos como candidatos, pero no sustituyen la evaluación propia, porque difieren en versión del corpus, partición, tratamiento de duplicados y clase positiva. **Los resultados que siguen son propios** y proceden de la ejecución documentada del notebook (sección 3.4).

4.2 Resultados obtenidos del Proyecto en base a los objetivos

RESULTADO OE1: diagnóstico del problema, caracterización del dataset y referencia de evaluación

Diagnóstico del problema. La revisión de evidencia (secciones I y 1.1) documentó el impacto económico internacional de las estafas por SMS (FTC, 2025), el aumento de las denuncias por fraude informático en el Perú (Diario Oficial El Peruano, 2024) y un antecedente contextual jurídico reciente en Trujillo (Lulli Caceres y Ortiz Gutierrez, 2024). Se identificaron siete antecedentes internacionales, dos nacionales y uno local contextual. No se localizó una estadística reciente desagregada para el canal SMS ni se verificó un clasificador local de spam SMS; ambos requisitos permanecen pendientes.

Caracterización del dataset. El archivo oficial contiene 5574 mensajes: 4827 legítimos (86.6 %) y 747 spam (13.4 %) (Figura 4). No hay mensajes vacíos ni etiquetas contradictorias, pero sí 403 duplicados exactos: 94 mensajes spam (12.6 % del spam) y 309 legítimos (6.4 %) repiten un texto previo, de modo que la proporción de duplicados es mayor en el spam. Los mensajes spam son casi el doble de largos (mediana de 149 frente a 52 caracteres) y concentran las secuencias largas de dígitos, los enlaces y los símbolos monetarios (Tabla 8).

Referencia de evaluación. Un clasificador que predice siempre la clase mayoritaria obtiene en validación una *accuracy* de 0.866 y un F1 de spam de 0 (0 verdaderos positivos y 149 falsos negativos), y en prueba una *accuracy* de 0.865. Esta línea base demuestra que la *accuracy* sola no informa sobre la detección de spam y fija la referencia mínima que cualquier modelo debe superar en F1 de spam.

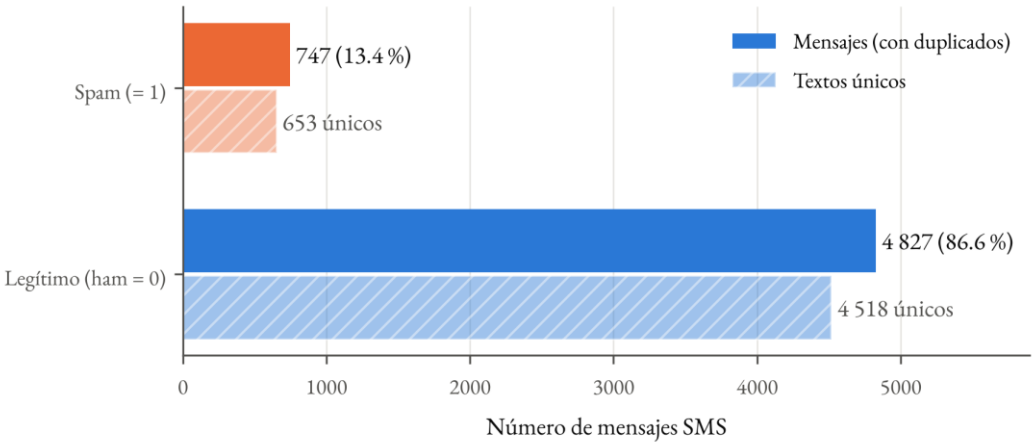


Figura 4

Distribución de mensajes por clase en el SMS Spam Collection v.1 (con y sin duplicados exactos)

Elaboración propia a partir del archivo oficial de UCI (diagnostico_dataset.json).

RESULTADO OE2: diseño e implementación de la solución

Se diseñó y ejecutó el *pipeline* de la sección 3.6. La búsqueda de hiperparámetros eligió C = 10 con unigramas y bigramas para ambos algoritmos; en validación, la SVM lineal calibrada superó a la regresión logística en F1 de spam (0.948 frente a 0.921; diferencia de 0.026, mayor que el margen de 0.005), por lo que fue seleccionada, y la regla de umbral eligió $\tau = 0.70$ (sección 4.3). El Listado 4 contiene el bloque central del algoritmo principal, **ejecutado** en el notebook el 18 de octubre de 2026; no es código ilustrativo.

Listado 4

Núcleo del algoritmo principal: pipeline, calibración y regla de decisión (código ejecutado)

```
TOKEN_PATTERN = r"(?u)\b\w+\b|[£$€!?%&]"

def make_pipeline(algorithm, C=1.0, ngram_range=(1, 1)):
  vec = TfidfVectorizer(lowercase=True, token_pattern=TOKEN_PATTERN, ngram_range=ngram_range,
                        min_df=1, sublinear_tf=False, norm="l2", use_idf=True, smooth_idf=True)
  if algorithm == "logreg":
    clf = LogisticRegression(C=C, solver="lbfgs", max_iter=2000)
  else:  # "linsvm"
    clf = LinearSVC(C=C, loss="squared_hinge", dual="auto", max_iter=20000, random_state=42)
  return Pipeline([("tfidf", vec), ("clf", clf)])

def make_calibrated_svm(C, ngram_range, cv_splits):
  # cv_splits: 5 pliegues agrupados calculados SOLO sobre el entrenamiento
  base = make_pipeline("linsvm", C=C, ngram_range=ngram_range)
  return CalibratedClassifierCV(estimator=base, method="sigmoid", cv=cv_splits, ensemble=True)

def decide(p_spam, tau):
  p_spam = np.asarray(p_spam, dtype=float)
  y_hat = (p_spam > 0.5).astype(int)                  # empate p = 0.5 -> ham
  accepted = np.maximum(p_spam, 1.0 - p_spam) >= tau      # c < tau -> revisión
  return y_hat, accepted

modelo = make_calibrated_svm(10.0, (1, 2), cv_splits_tr).fit(X_train, y_train)
```

Extracto de src/spam_sms/modeling.py, src/spam_sms/decision.py y de la celda 8 del notebook; se omitieron importaciones y comentarios.

La aplicación web se ejecutó localmente en el entorno de trabajo y se recorrió con un navegador automatizado (Playwright) usando mensajes de demostración escritos para la prueba, no tomados del corpus. Un mensaje del tipo «You have been selected for a special offer, reply to this number» obtuvo P(spam) = 0.576 y confianza 0.576 < τ, por lo que la aplicación no decidió y lo envió a la bandeja (Figura 5). En la bandeja, un revisor simulado («Revisor demo»), introducido por el propio guion de recorrido, confirmó la etiqueta spam y la revisión quedó registrada en SQLite (Figura 6). Las capturas del formulario y de las decisiones

automáticas se incluyen en el Anexo G. Las cuatro pruebas unitarias del núcleo —regla de empate y umbral, métricas selectivas, selección de τ, y carga y partición sin fuga— se superaron.

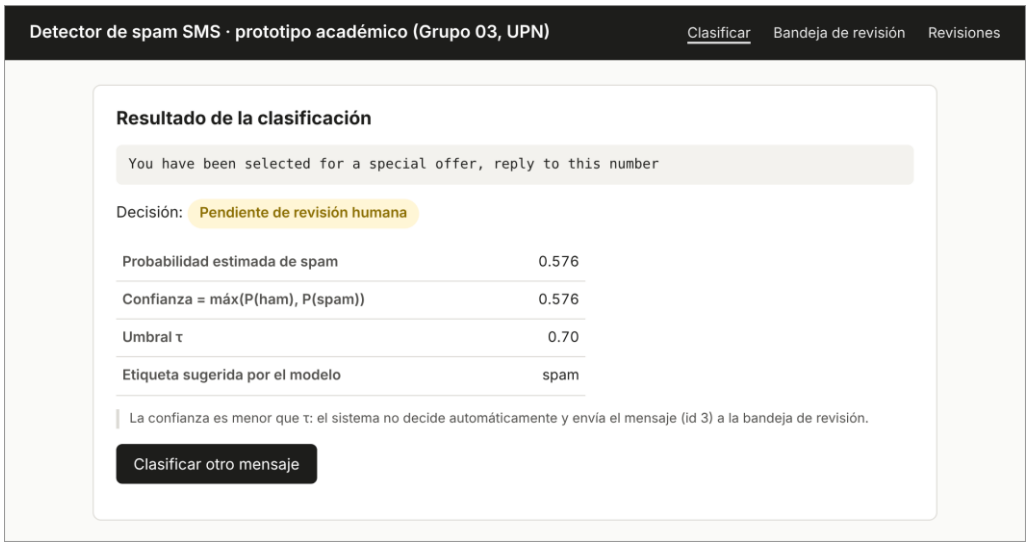


Figura 5

Resultado de un mensaje enviado a revisión humana en el prototipo

Captura de la aplicación Flask en ejecución local (18-10-2026), obtenida con Playwright; mensaje de demostración.

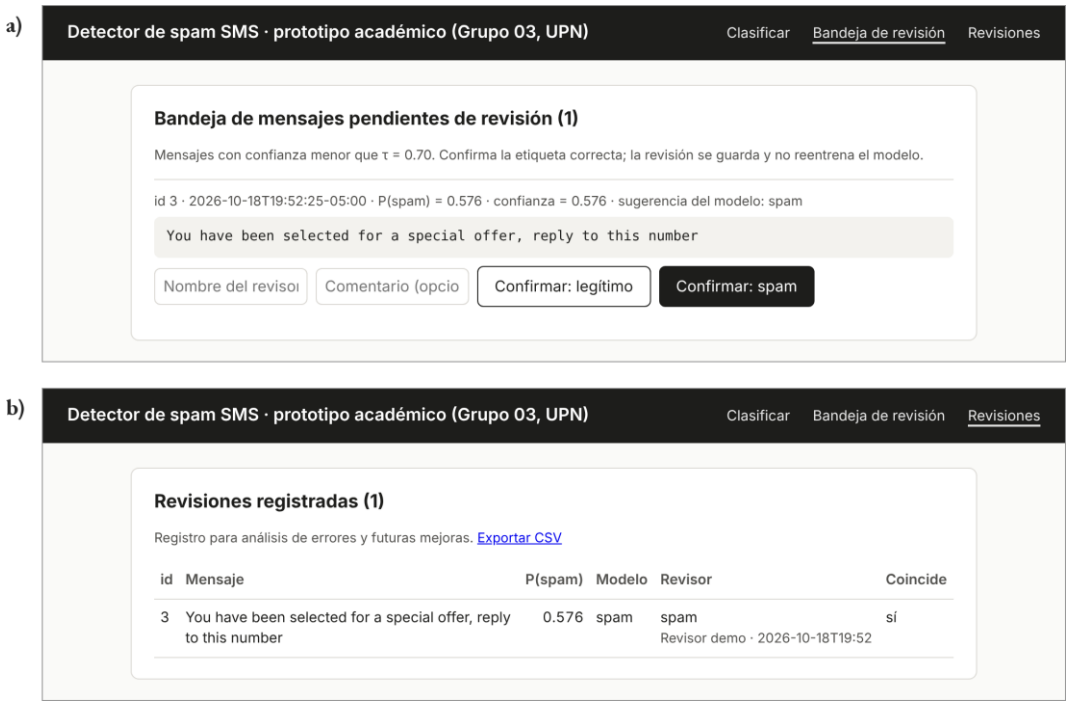


Figura 6

Bandeja de pendientes (a) y registro de la revisión confirmada (b)

Capturas de la aplicación Flask en ejecución local (18-10-2026). La revisión fue realizada en la demostración, no por un usuario real.

Qué se demostró y qué no. Se demostró que el *pipeline* congelado y la regla de abstención funcionan dentro de una aplicación que guarda decisiones y revisiones. No se demostró el uso por revisores reales, el despliegue en un servidor, la seguridad de acceso ni el reentrenamiento; guardar revisiones no implica que el modelo aprenda de ellas.

RESULTADO OE3: evaluación y análisis cuantitativo

En la partición de prueba, evaluada con el modelo congelado, sin modificar las decisiones tras observarla, la SVM lineal calibrada obtuvo precision de spam 0.986, recall 0.913, F1 0.948 y *accuracy* 0.987, con 2 falsos

positivos y 13 falsos negativos entre 1115 mensajes; ambas metas del proyecto se cumplieron (sección 4.3). Con $\tau = 0.70$, el sistema resolvió automáticamente 1109 mensajes (cobertura 99.46 %) y envió 6 a revisión (0.54 %); el error entre los aceptados fue 0.99 %, frente a 1.35 % sin abstención. De los 6 mensajes enviados a revisión, 4 habrían sido errores si se hubieran decidido automáticamente; en cambio, 11 errores —todos falsos negativos— se aceptaron con confianza ≥ 0.70, siete de ellos con confianza ≥ 0.98. Así, para este modelo y este corpus, la revisión humana reduce el error de las decisiones automáticas a un costo bajo de revisión, pero no puede corregir los errores emitidos con seguridad.

RESULTADO OG: síntesis respecto del objetivo general

Respecto del **funcionamiento del software**, se aplicaron dos algoritmos supervisados y se incorporó un mecanismo de revisión que opera de extremo a extremo en un prototipo local. Respecto de la **eficacia predictiva**, el modelo seleccionado alcanzó en prueba F1 de spam 0.948 (IC 95 %: 0.920–0.972) y precision 0.986 (IC 95 %: 0.963–1.000) en el corpus de referencia, y la abstención mostró un compromiso favorable pero acotado entre cobertura y error. El objetivo general se cumple para el alcance definido (mensajes en inglés del corpus de UCI, revisión simulada); su validez para usuarios peruanos, mensajes en español, fraudes recientes y revisores reales no está demostrada.

4.3 Ejecución, métricas y análisis de datos

Selección de hiperparámetros. La Tabla 12 y la Figura 7 presentan las doce configuraciones evaluadas por validación cruzada agrupada de cinco pliegues en el entrenamiento. La regresión logística con C = 0.1 predijo sólo la clase mayoritaria (F1 = 0): con regularización fuerte y 13.4 % de spam, sus probabilidades nunca superaron 0.5. Para ambos algoritmos, la mejor configuración fue C = 10 con unigramas y bigramas, que es el **límite superior de la rejilla**; por ello no puede descartarse que valores mayores de C mejoren el resultado, aspecto que se retoma en las recomendaciones. La búsqueda tardó 7.3 s para la regresión logística y 4.5 s para la SVM.

Tabla 12

Validación cruzada agrupada de 5 pliegues en el entrenamiento (F1, precision y recall de spam)

Algoritmo	C	n-gramas	F1 CV (media ± DE)	Precision CV	Recall CV	F1 de ajuste
Regresión logística	0.1	(1, 1)	0.000 ± 0.000	0.000	0.000	0.000
Regresión logística	0.1	(1, 2)	0.000 ± 0.000	0.000	0.000	0.000
Regresión logística	1	(1, 1)	0.868 ± 0.028	0.997	0.770	0.889
Regresión logística	1	(1, 2)	0.832 ± 0.031	1.000	0.714	0.819
Regresión logística	10	(1, 1)	0.943 ± 0.015	0.986	0.904	0.998
Regresión logística	**10**	**(1, 2)**	**0.948 ± 0.012**	**0.998**	**0.904**	**0.999**
SVM lineal	0.1	(1, 1)	0.898 ± 0.013	0.995	0.819	0.920
SVM lineal	0.1	(1, 2)	0.868 ± 0.031	1.000	0.768	0.866
SVM lineal	1	(1, 1)	0.946 ± 0.015	0.988	0.908	0.998
SVM lineal	1	(1, 2)	0.957 ± 0.016	0.993	0.924	1.000
SVM lineal	10	(1, 1)	0.944 ± 0.017	0.978	0.913	1.000
SVM lineal	**10**	**(1, 2)**	**0.960 ± 0.013**	**0.991**	**0.931**	**1.000**

Nota. En negrita, la mejor configuración de cada algoritmo. La SVM se evaluó aquí sin calibrar (umbral 0 de la función de decisión). DE = desviación estándar entre pliegues. F1 de ajuste = F1 sobre los pliegues usados para entrenar.

Elaboración propia a partir de cv_rejilla.csv.

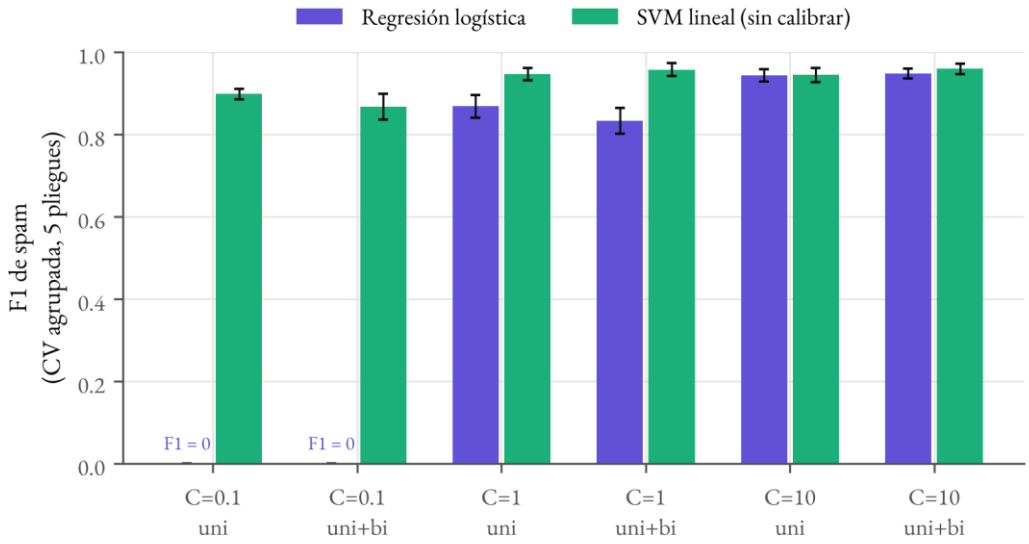


Figura 7

F1 de spam por configuración en la validación cruzada agrupada del entrenamiento

Elaboración propia a partir de cv_rejilla.csv. Las barras de error indican ± 1 desviación estándar entre los cinco pliegues.

Fuga por duplicados. Al repetir la validación cruzada de la mejor configuración con pliegues no agrupados, el F1 medio fue 0.944 para la regresión logística (0.948 con agrupación) y 0.960 para la SVM (igual que con agrupación). En esta muestra, por lo tanto, no se observó un optimismo apreciable atribuible a los duplicados; la partición agrupada se mantiene como control de diseño, no porque haya modificado las conclusiones.

Curvas de aprendizaje y sobreajuste. La Figura 8 muestra que el F1 de ajuste es prácticamente 1.0 en todos los tamaños, mientras que el F1 en los pliegues excluidos crece con los datos: de 0.595 a 0.948 en la regresión logística y de 0.849 a 0.960 en la SVM. La brecha final, de 0.05 en la regresión logística y de 0.04 en la SVM, indica varianza o sobreajuste moderado propio de modelos con decenas de miles de términos; la curva de la SVM se estabiliza desde unos 2000 mensajes y la de la regresión logística todavía crece, de modo que más datos etiquetados podrían beneficiar sobre todo a esta última.

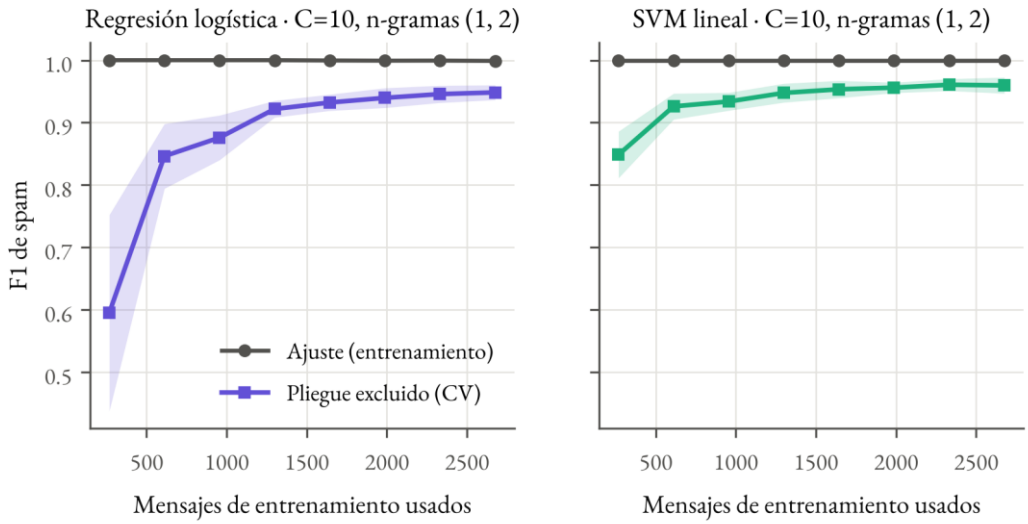


Figura 8

Curvas de aprendizaje de la mejor configuración de cada algoritmo

Elaboración propia a partir de curva_aprendizaje_logreg.csv y curva_aprendizaje_linsvm.csv (entrenamiento, CV agrupada de 5 pliegues). Bandas: ± 1 desviación estándar.

Calibración. En validación, la SVM calibrada tuvo menor puntuación de Brier, menor pérdida logarítmica y menor ECE que la regresión logística (Tabla 13). La Figura 9 muestra, sin embargo, que la mayoría de los mensajes recibe probabilidades extremas: los intervalos intermedios (entre 0.1 y 0.9) contienen entre 1 y 12 mensajes en la SVM y entre 2 y 30 en la regresión logística, por lo que la forma de la curva en esa zona es inestable y no debe sobreinterpretarse.

Tabla 13

Calidad de las probabilidades en validación (n = 1115)

Modelo	Brier	Pérdida logarítmica	ECE (10 intervalos)
Regresión logística (C = 10, (1, 2))	0.0151	0.0660	0.0225
SVM lineal calibrada (C = 10, (1, 2))	0.0099	0.0406	0.0069

Elaboración propia a partir de calibracion_validacion.csv.

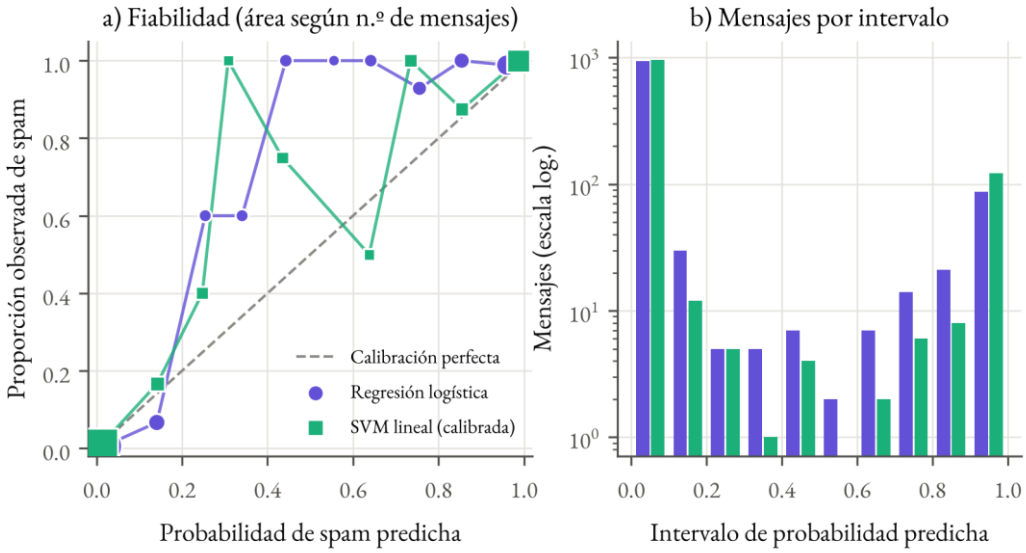


Figura 9

Diagrama de fiabilidad (a) y número de mensajes por intervalo de probabilidad (b) en validación

Elaboración propia a partir de fiabilidad_validacion_*.csv. El área de cada marcador crece con el número de mensajes del intervalo.

Comparación de modelos en validación. La Tabla 14 resume la comparación preregistrada. Ambos modelos superaron con claridad a la línea base (H1 se cumple). Con el mismo número de falsos positivos, la SVM calibrada redujo los falsos negativos de 20 a 13, y fue seleccionada por su F1.

Tabla 14

Comparación en validación con umbral 0.5, sin abstención (n = 1115; 149 spam)

Modelo	VP	FP	FN	VN	Precision	Recall	F1	Accuracy
Línea base (clase mayoritaria)	0	0	149	966	—	0.000	0.000	0.866
Regresión logística	129	2	20	964	0.985	0.866	0.921	0.980
SVM lineal calibrada (seleccionada)	136	2	13	964	0.986	0.913	**0.948**	0.987

Nota. VP, FP, FN y VN con spam como clase positiva. La precision de la línea base no está definida (no predice spam).

Elaboración propia a partir de comparacion_validacion.csv y linea_base_validacion.json.

Evaluación de umbrales en validación. La Tabla 15 y la Figura 10 describen el compromiso. En la SVM calibrada, subir τ de 0.50 a 0.95 redujo el error entre aceptados de 1.35 % a 0.29 % y elevó la tasa de revisión de 0 a 6.55 %. Los umbrales 0.70, 0.80, 0.90 y 0.95 cumplieron ambas restricciones, y la regla eligió $\tau = 0.70$ por ser el de mayor cobertura (99.37 %). En la regresión logística, el mismo error se alcanza con más

revisión, porque sus probabilidades son menos extremas en los casos difíciles: con $\tau = 0.95$ habría que revisar el 16.6 % de los mensajes. H2 se cumple en validación.

Tabla 15

Clasificación selectiva en validación según el umbral τ (n = 1115)

τ	Aceptados	Revisión	Cobertura	Errores aceptados	Error aceptados	Precision acept.	Recall acept.
SVM lineal calibrada (modelo seleccionado)							
0.50	1115	0	100.00 %	15	1.35 %	0.986	0.913
0.60	1111	4	99.64 %	12	1.08 %	0.986	0.932
0.70	**1108**	**7**	**99.37 %**	**10**	**0.90 %**	**0.993**	**0.938**
0.80	1097	18	98.39 %	8	0.73 %	0.992	0.949
0.90	1077	38	96.59 %	5	0.46 %	1.000	0.961
0.95	1042	73	93.45 %	3	0.29 %	1.000	0.973
Regresión logística							
0.50	1115	0	100.00 %	22	1.97 %	0.985	0.866
0.60	1106	9	99.19 %	15	1.36 %	0.984	0.907
0.70	1094	21	98.12 %	12	1.10 %	0.984	0.923
0.80	1075	40	96.41 %	8	0.74 %	0.991	0.939
0.90	1024	91	91.84 %	6	0.59 %	0.989	0.945
0.95	930	185	83.41 %	1	0.11 %	1.000	0.982

Nota. Precision y recall de spam calculados sólo sobre los mensajes aceptados. En negrita, el umbral elegido por la regla preregistrada. Elaboración propia a partir de umbrales_validacion_linsvm.csv y umbrales_validacion_logreg.csv.

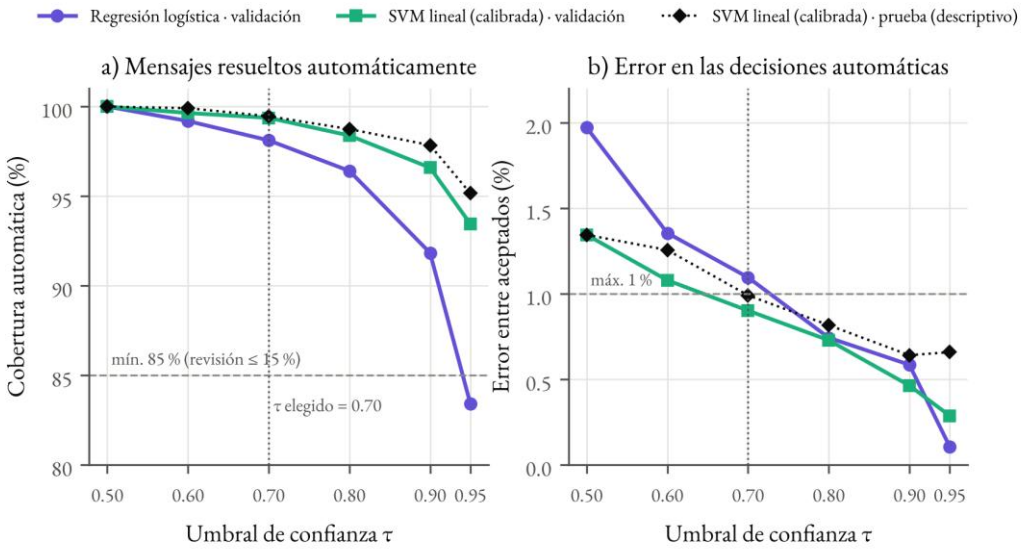


Figura 10

Cobertura automática (a) y error entre mensajes aceptados (b) según el umbral τ

Elaboración propia a partir de umbrales_validacion_*.csv y umbrales_prueba_descriptivo.csv. La curva de prueba es descriptiva y no se usó para elegir τ.

Evaluación final en prueba. La Tabla 16 presenta las métricas del modelo congelado con sus intervalos de confianza. Las metas preestablecidas se cumplen incluso en el límite inferior de los intervalos (F1 ≥ 0.920 y precision ≥ 0.963), por lo que H3 se cumple. La Figura 11 muestra las matrices de confusión de la prueba

completa y del subconjunto aceptado con $\tau = 0.70$. Las métricas de prueba son prácticamente iguales a las de validación, lo que no sugiere un sobreajuste a la partición de validación.

Tabla 16

Evaluación final del modelo congelado en la partición de prueba (n = 1115; 150 spam)

Métrica (spam como positiva)	Valor	IC 95 % bootstrap	Meta
Precision	0.986	0.963–1.000	≥ 0.95 (cumple)
Recall	0.913	0.865–0.956	—
F1	0.948	0.920–0.972	≥ 0.90 (cumple)
Accuracy (complementaria)	0.987	0.979–0.993	—
VP / FP / FN / VN	137 / 2 / 13 / 963	—	—
Brier / ECE	0.0107 / 0.0075	—	—
Línea base: accuracy / F1	0.865 / 0.000	—	—
Sensibilidad: prueba sin duplicados internos (n = 1032): precision / recall / F1	0.983 / 0.899 / 0.939	—	—

Elaboración propia a partir de evaluacion_final_prueba.json (2000 remuestreos, semilla 42).

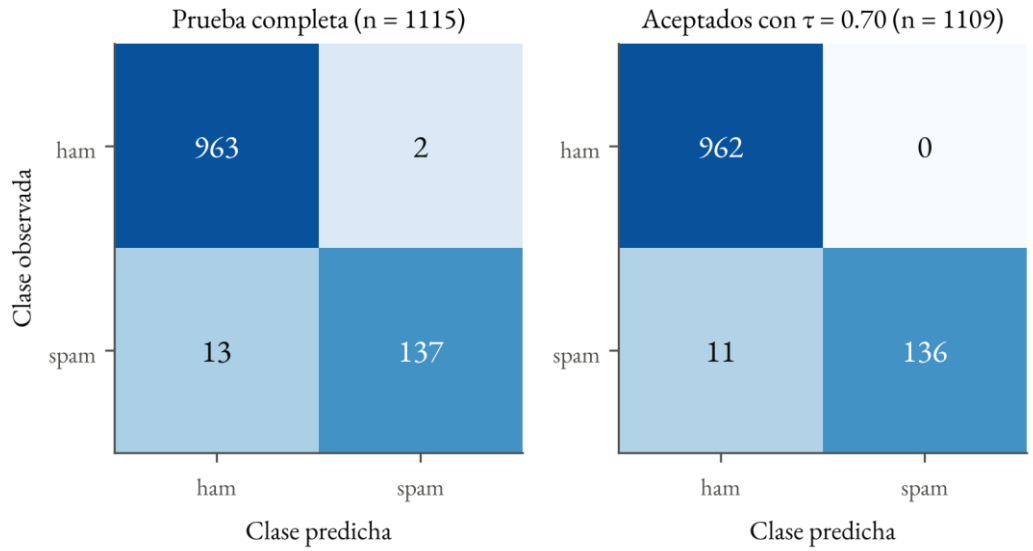


Figura 11

Matrices de confusión en prueba: cobertura total y mensajes aceptados con $\tau = 0.70$

Elaboración propia a partir de evaluacion_final_prueba.json. Escala de color logarítmica.

Clasificación selectiva en prueba. La Tabla 17 detalla el tamaño y la composición del subconjunto aceptado. Los 6 mensajes enviados a revisión se repartieron por igual entre clases (3 legítimos y 3 spam), y el subconjunto aceptado conservó una proporción de spam similar a la de la prueba (13.3 % frente a 13.5 %). En los aceptados no hubo falsos positivos: los dos falsos positivos de la prueba quedaron en revisión. La mejora del error entre aceptados, de 1.35 % a 0.99 %, **no es una mejora global** mientras los seis mensajes no se revisen. Sólo bajo el supuesto de un revisor perfecto —una simulación que usa las etiquetas del corpus— el sistema completo tendría precision 1.000, recall 0.927 y F1 0.962.

Tabla 17

Clasificación selectiva en prueba con $\tau = 0.70$

Indicador	Valor
Mensajes aceptados (cobertura) / en revisión (tasa)	1109 (99.46 %) / 6 (0.54 %)
Composición de aceptados: ham / spam	962 / 147 (13.3 % spam)
Composición en revisión: ham / spam	3 / 3
Errores entre aceptados (FP / FN)	11 (0 / 11) = 0.99 %
Errores evitados: en revisión que habrían sido errores (FP / FN)	4 (2 / 2)
Aceptados: precision / recall / F1 de spam	1.000 / 0.925 / 0.961
Aceptados: precision / recall / F1 de ham	0.989 / 1.000 / 0.994
Sistema con revisión ideal **simulada**: VP / FP / FN / VN	139 / 0 / 11 / 965
Sistema con revisión ideal **simulada**: precision / recall / F1	1.000 / 0.927 / 0.962

Elaboración propia a partir de evaluacion_final_prueba.json. La revisión ideal simulada no es una prueba con usuarios.

Distribución de probabilidades y análisis de errores. La Figura 12 muestra que el modelo asigna a casi todos los mensajes probabilidades cercanas a 0 o a 1 y que la zona de revisión ($0.30 < p < 0.70$) contiene muy pocos casos; por eso, con $\tau = 0.70$, la tasa de revisión es baja. La Tabla 18 enumera los 15 errores de prueba. Ninguno de los 13 falsos negativos contiene un enlace y sólo 2 contienen una secuencia de cinco o más dígitos, es decir, carecen de los marcadores que el modelo aprendió como típicos del spam. Varios relatan o denuncian spam recibido (quejas por cobros o por mensajes de tarifa premium), otros son chistes o mensajes de cadena, y uno es una subasta sin enlace. Estos casos sugieren ambigüedad en el criterio de etiquetado del corpus —un mensaje que se queja de spam no es, en sí mismo, spam— más que un fallo aleatorio, y explican por qué el modelo se equivoca con confianza. Los dos falsos positivos, ambos enviados a revisión, son mensajes legítimos breves o con mezcla de idiomas.

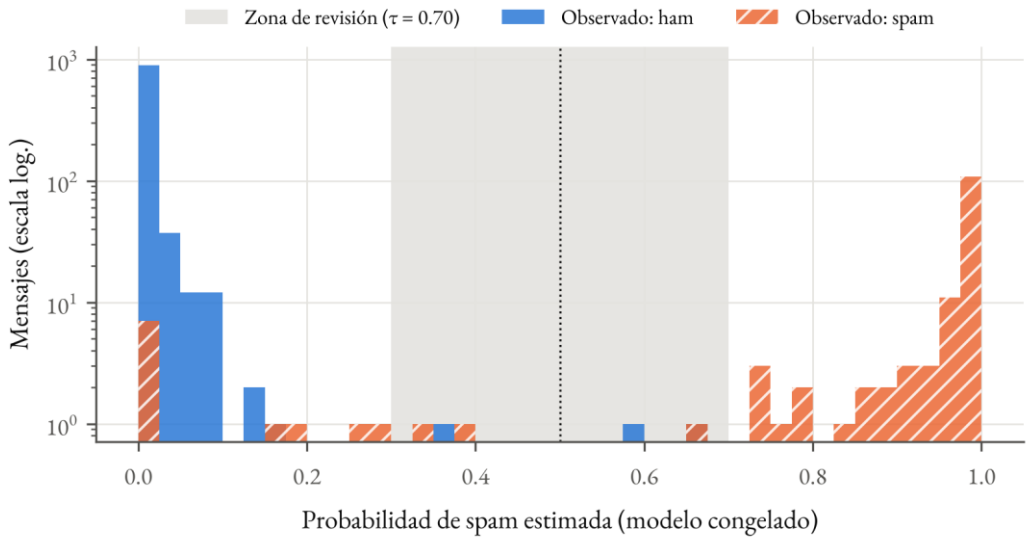


Figura 12

Distribución de la probabilidad de spam estimada en prueba, por clase observada

Elaboración propia a partir de las predicciones del modelo congelado en la partición de prueba. Eje vertical en escala logarítmica.

Tabla 18

Errores del modelo congelado en la partición de prueba

Línea	Tipo	P(spam)	Confianza	Destino con τ = 0.70	Inicio del mensaje (truncado)
4514	FN	0.001	0.999	Automático	Money i have won wining number 946 wot do i do next
869	FN	0.002	0.998	Automático	Hello. We need some posh birds and chaps to user trial prods...
751	FN	0.002	0.998	Automático	Do you realize that in about 40 years, we'll have thousands...
68	FN	0.009	0.991	Automático	Did you hear about the new "Divorce Barbie"? It comes with...
3460	FN	0.011	0.989	Automático	Not heard from U4 a while. Call me now am here all night...
3419	FN	0.014	0.986	Automático	LIFE has never been this much fun and great until you came in...
2247	FN	0.016	0.984	Automático	Hi ya babe x u 4goten bout me?' scammers getting smart...
731	FN	0.152	0.848	Automático	Email AlertFrom: Jeri StewartSize: 2KBSubject: Low-cost...
955	FN	0.193	0.807	Automático	Filthy stories and GIRLS waiting for your
5379	FN	0.259	0.741	Automático	The current leading bid is 151. To pause this auction send...
4069	FN	0.300	0.700	Automático	TBS/PERSOLVO. been chasing us since Sept for£38 definitely...
1460	FN	0.348	0.652	Revisión	Bought one ringtone and now getting texts costing 3 pound...
5542	FN	0.389	0.611	Revisión	ASKED 3MOBILE IF 0870 CHATLINES INCLU IN FREE MINS...
84	FP	0.586	0.586	Revisión	Yup next stop.
4488	FP	0.653	0.653	Revisión	Miss call miss call khelate kintu opponenter miss call...

Nota. Línea = posición del mensaje en el archivo original, contando desde 0. FN = spam clasificado como legítimo; FP = legítimo clasificado como spam. El mensaje 4069 tiene confianza 0.7003, apenas sobre el umbral.

Elaboración propia a partir de errores_prueba.csv. Los textos pertenecen al corpus público de UCI.

Sondas exploratorias. Para ilustrar los límites de generalización, se clasificaron seis mensajes escritos para la demostración (Tabla 19). Un aviso de entrega fallida con solicitud de datos —la modalidad más frecuente de 2024 según la FTC (2025)— se clasificó como legítimo con confianza 0.974, y un recordatorio legítimo de cita con la fórmula «Reply STOP» se clasificó como spam con confianza 0.958. Seis ejemplos no constituyen una evaluación ni permiten estimar tasas de error; sólo ilustran fallos plausibles ante mensajes redactados con patrones actuales y formatos legítimos que el modelo asocia con spam.

Tabla 19

Sondas exploratorias con mensajes de demostración (no constituyen evaluación)

Tipo de sonda	Mensaje	P(spam)	Decisión
Conversación personal	Are we still meeting for lunch tomorrow at 1pm?	0.002	Legítimo
Premio clásico	Congratulations! You have won a 500 GBP voucher. Text CLAIM to 80086 now...	0.997	Spam
Oferta ambigua	You have been selected for a special offer, reply to this number	0.576	Pendiente de revisión
Entrega fallida (patrón actual)	Your package could not be delivered. Please confirm your details at the link we sent	0.026	Legítimo (error plausible)
Suplantación bancaria sin enlace	Hi, this is the bank. Please call us back about your account today	0.188	Legítimo (dudoso)
Notificación legítima con STOP	Reminder: your appointment is tomorrow. Reply STOP to cancel	0.958	Spam (error plausible)

Elaboración propia a partir de sondas_exploratorias.csv; mensajes redactados para la demostración, sin etiqueta de referencia.

Costo computacional. En el entorno de ejecución, el ajuste final tardó 0.42 s para la regresión logística y 0.69 s para la SVM calibrada (cinco modelos), y la predicción de un mensaje individual tuvo una mediana

de 5.9 ms (percentil 95: 8.6 ms; 300 mensajes). Estos tiempos son compatibles con una aplicación interactiva, aunque deben medirse de nuevo en los equipos del grupo.

V. DISCUSIÓN Y CONCLUSIONES

5.1 Discusión de resultados con los antecedentes

La Tabla 20 contrasta las métricas de la sección 4.3 con las de los antecedentes. El contraste es **descriptivo**: sólo se considera comparable una cifra cuando coinciden el corpus, la clase positiva y la métrica, y aun así las diferencias de protocolo impiden concluir que un método sea superior.

Tabla 20

Contraste entre los resultados propios y los antecedentes

Antecedente	Corpus y protocolo	Resultado reportado	Resultado propio comparable	Lectura del contraste
Şahin et al. (2026)	UCI, 5574; CV 10 pliegues sobre todo el corpus; trigramas de caracteres + RL; spam positiva	P 98.55 %; R 90.50 %; F1 94.32 %	Prueba: P 98.6 %; R 91.3 %; F1 94.8 %	Magnitud similar; protocolo, representación y tratamiento de duplicados distintos
Ghourabi y Alohaly (2023)	UCI, 5574; una partición 80/20; duplicados no reportados	SVM + TF-IDF: Acc 97.85 %; conjunto GPT-3: Acc 99.91 %	Acc 98.65 % (complementaria)	Accuracy poco informativa con desbalance; partición única sin agrupar
Altunay y Albayrak (2024)	Turco + inglés (10 325); CNN + GRU	Acc 99.08 %; F1 99.22 %	No comparable	Otra mezcla de datos e idiomas
Maqsood et al. (2023)	Kaggle, 5000 SMS; ~67.7/32.3	SVM: Acc 99.6 %	No comparable	Versión distinta del corpus; inconsistencias descriptivas
Abdel-Jaber (2025)	Kaggle, 5572; 10 particiones; ham positiva	SVM: F1 98.52 % (ham)	F1 de ham en prueba: 99.2 %	Sólo comparable invirtiendo la clase; muestreo y protocolo distintos
Bani Younes y Ababneh (2026)	UCI 5100 + prueba externa (Kaggle)	Entre dominios: P y R 45–65 %	Sondas exploratorias; sin prueba externa	Advierte la caída fuera del corpus; evaluación externa pendiente
Salman et al. (2024)	67 018 SMS; prueba convencional y conjunto adversarial separado	TCSVM + Word2Vec: F1 99 % (Tabla 12); en conjunto adversarial, F1 91 % sin ataque y 21 % con espacios (Tabla 14)	Sin evaluación adversarial	Evasión verificada en su protocolo; evaluación adversarial propia pendiente
Calero Sinche et al. (2024)	Revisión: 36 estudios de *smishing*	RF, árboles y redes neuronales más usados	No aplica	Nuestro modelo no es específico de *smishing*

Nota. P = precision; R = recall; Acc = accuracy; RL = regresión logística.

Elaboración propia a partir de la Tabla 1 y de evaluacion_final_prueba.json.

El antecedente más cercano, Şahin et al. (2026), obtiene con regresión logística y trigramas de caracteres un F1 de spam de 94.32 %, del mismo orden que nuestro 94.8 % con SVM calibrada y n-gramas de palabras. Ambos estudios coinciden en el patrón de error: una precision cercana a 98.5 % y un recall de alrededor de 91 %, es decir, pocos mensajes legítimos marcados como spam y una fracción de spam no detectada. En nuestro análisis de errores, parte del spam no detectado carece de los marcadores habituales; queda pendiente evaluar si otras representaciones o modelos recuperan esos casos. Hay dos diferencias de protocolo relevantes. Ellos promedian diez pliegues sobre todo el corpus, mientras que nosotros informamos una sola partición de prueba retenida, por lo que nuestro intervalo (0.920–0.972) refleja la variabilidad de 1115 mensajes. Además, ellos afirman que el corpus no tiene duplicados, mientras que nuestro diagnóstico del

mismo archivo encontró 403; en nuestro caso la agrupación no cambió de forma apreciable el F1 de validación cruzada, pero la discrepancia muestra la importancia de verificar el archivo en lugar de asumir sus propiedades.

Las accuracy de 99 % o más que informan Ghourabi y Alohaly (2023), Altunay y Albayrak (2024) y Maqsood et al. (2023) no demuestran la superioridad de esos métodos sobre el nuestro: provienen de particiones únicas sin agrupar duplicados, de mezclas de idiomas o de versiones distintas del corpus, y la accuracy está dominada por la clase mayoritaria —nuestra propia línea base alcanza 0.865—. El caso de Abdel-Jaber (2025) ilustra otro problema de comparabilidad: con spam codificado como 0, sus métricas corresponden a la clase legítima; medido de la misma forma, nuestro F1 de la clase legítima en prueba es 0.992, pero ni el muestreo ni el protocolo son equivalentes. El tamaño de 5572 mensajes coincide con el resultado de la lectura con comillado por defecto documentada en la sección 3.5; esta coincidencia no permite determinar cómo se produjo la versión utilizada por ese estudio.

Bani Younes y Ababneh (2026) y Salman et al. (2024) advierten que el desempeño cambia con la fuente, la época y la adaptación del contenido. En el conjunto adversarial separado de Salman et al., TCSVM con Word2Vec pasó de F1 91 % sin modificación a 21 % con inserción de espacios (Tabla 14, p. 24317); estas cifras corresponden a ese mismo conjunto adversarial y no a su prueba convencional de la Tabla 12. Nuestras sondas exploratorias son coherentes con ese riesgo —un aviso de entrega fallida, modalidad frecuente según la FTC (2025), se clasificó como legítimo—, pero seis mensajes no constituyen una evaluación. La comparación cuantitativa propia con una prueba externa o adversarial permanece pendiente.

Respecto de la revisión humana, ninguno de los antecedentes informa cobertura, abstención ni calibración, por lo que la comparación cuantitativa de los resultados selectivos con la literatura de spam SMS **no es posible** con la evidencia disponible. En términos de los marcos teóricos, el comportamiento observado coincide con el rechazo por ambigüedad descrito por Hendrickx et al. (2024): la abstención captura los errores cercanos a la frontera —4 de los 6 mensajes revisados habrían sido errores—, pero no los errores alejados de ella. Para estudiar esos errores conviene evaluar el rechazo por novedad o señales adicionales con entrenamiento y validación; la tabla descriptiva de prueba muestra que, aun con $\tau = 0.95$, siete falsos negativos seguían aceptados.

5.2 Conclusiones por objetivo

OG. Se aplicaron y evaluaron dos algoritmos supervisados y se incorporó un mecanismo de revisión humana de predicciones inciertas. En el corpus SMS Spam Collection v.1, la SVM lineal calibrada alcanzó en prueba F1 de spam 0.948 y precision 0.986, por encima de las metas, y con $\tau = 0.70$ redujo el error de las decisiones automáticas de 1.35 % a 0.99 % enviando a revisión el 0.54 % de los mensajes. La respuesta a la pregunta de investigación es, por tanto, que la revisión de predicciones inciertas mejora el error de lo automatizado con un costo de revisión bajo, pero con un límite: los errores emitidos con alta confianza —11 de 15 en prueba— no se ven afectados. El objetivo se cumple dentro del alcance definido; no se ha demostrado para mensajes en español, fraudes recientes ni revisores reales.

OE1 (concluido, con pendientes). Se documentó el problema en los niveles internacional y nacional y se incorporó un antecedente contextual jurídico de Trujillo; se caracterizó el archivo oficial (5574 mensajes, 13.4 % spam, 403 duplicados exactos, sin vacíos ni contradicciones), se detectó y corrigió un error de lectura que reduce el corpus a 5572 filas y se estableció la línea base (accuracy 0.866; F1 de spam 0). Permanecen pendientes un antecedente metodológico local de clasificación de spam SMS y una estadística reciente del canal desagregada para el Perú y Trujillo.

OE2 (concluido a nivel de prototipo). Se implementaron el *pipeline* TF-IDF con SVM lineal calibrada, la regla de abstención con desempate determinista y una aplicación Flask con bandeja de revisión y registro en SQLite, verificados con cuatro pruebas unitarias y un recorrido automatizado. Quedan pendientes el repositorio con historial de autoría, la ejecución en los equipos del grupo, la autenticación, la política de retención y anonimización y el despliegue.

OE3 (concluido para el corpus de referencia). Se cuantificó el compromiso entre cobertura y error en validación y en prueba, con intervalos de confianza y análisis de errores. Los objetivos de evaluación con revisores humanos, con datos externos o en español **no están concluidos** y requieren nuevos datos.

5.3 Limitaciones y recomendaciones técnicas

a) Limitaciones

- **Datos.** Se usó un único corpus público, con fuentes heterogéneas —un foro británico, una tesis, un corpus universitario de Singapur— y etiquetas que en algunos casos parecen ambiguas: hay mensajes que se quejan del spam etiquetados como spam. No se modificaron etiquetas.
- **Idioma.** Todos los mensajes están en inglés; el modelo no está validado para español ni para mensajes con mezcla de idiomas, como muestra un falso positivo en prueba.
- **Antigüedad del corpus. Los mensajes datan de alrededor de 2011 y no garantizan cobertura de los patrones actuales de fraude, como entregas fallidas, peajes o falsas alertas bancarias.**
- **Modelo.** Es un modelo lineal sobre n-gramas de palabras, sin señales del remitente, de los enlaces ni del contexto; C = 10 está en el límite de la rejilla; la calibración se estimó con pocos casos intermedios.
- **Cómputo.** La ejecución se realizó en un entorno en la nube con dos núcleos; los tiempos y la reproducción en los equipos del grupo están pendientes.
- **Validación. Hay una sola partición de prueba (1115 mensajes; 150 spam), sin validación externa ni revisores reales; los criterios no se preregistraron externamente; la prueba se recalculó en tres ejecuciones con resultados idénticos; y el código fue generado con asistencia de IA y está pendiente de revisión por el grupo. Los intervalos bootstrap remuestrean mensajes individuales; no incorporan explícitamente la dependencia entre duplicados del mismo grupo y son condicionales a esta única partición.**

b) Riesgos

La Tabla 21 identifica los riesgos técnicos principales y sus mitigaciones. Las valoraciones son estimaciones cualitativas del equipo, salvo donde se indica una frecuencia observada.

Tabla 21

Riesgos técnicos y mitigaciones

Riesgo	Descripción y evidencia	Mitigación concreta
Desbalance de clases	13.4 % de spam; la línea base logra 0.866 de accuracy con F1 = 0	Métricas de la clase spam, estratificación en todas las particiones; si se balancea, sólo en entrenamiento y con evaluación del aumento de FP
Sobreajuste	F1 de ajuste ≈ 1.0 frente a 0.96 en CV (brecha 0.04–0.05)	Regularización L2, selección por CV, curvas de aprendizaje; no reutilizar la prueba actual para nuevas elecciones
Fuga de información	Duplicados y preprocesamiento fuera del pliegue pueden inflar las métricas	Partición agrupada verificada (0 grupos compartidos), TF-IDF dentro del *pipeline*, congelamiento antes de la prueba
Privacidad	La aplicación guarda el texto completo de cada mensaje	Guardar sólo los pendientes, enmascarar números, enlaces y nombres, cifrar la base, limitar el acceso (sección VII)
Degradación del modelo	Corpus de 2011; las sondas sugieren fallos con patrones actuales	Vigilar la tasa de revisión y la distribución de probabilidades; reevaluar con datos recientes etiquetados; versionar el modelo
Evasión adversarial	Los emisores alteran la escritura para eludir filtros (Salman et al., 2024)	No exponer probabilidades a terceros; probar n-gramas de caracteres; pruebas con variantes ofuscadas
Errores de revisión humana	El revisor puede equivocarse; el sistema no lo detecta	Guía de etiquetado, doble revisión de una muestra, medir el acuerdo entre revisores
Dependencia de versiones	El artefacto Joblib depende de scikit-learn 1.9.1	Fijar versiones en `requirements.txt`; regenerar el modelo desde el notebook; verificar el SHA-256

Elaboración propia.

c) Desempeño del modelo

La Tabla 22 evalúa los criterios de la sección 1.4. Los casos de fallo observados son: (1) once falsos negativos aceptados con confianza ≥ 0.70, de mensajes sin enlaces y casi sin dígitos largos; (2) dos falsos positivos de mensajes legítimos breves o con mezcla de idiomas, ambos derivados a revisión; y (3) en las sondas, un aviso de entrega fallida clasificado como legítimo y una notificación con «Reply STOP» clasificada como spam. Verificaciones pendientes: reproducir el notebook en los equipos del grupo y comparar las huellas; evaluar con un conjunto externo reciente; construir un conjunto en español; y medir la calidad de revisores reales.

Tabla 22

Evaluación de los criterios de éxito e hipótesis

Criterio o hipótesis	Resultado	Estado
F1 de spam ≥ 0.90 en prueba (meta)	0.948 (IC 95 %: 0.920–0.972)	Cumple
Precision de spam ≥ 0.95 en prueba (meta)	0.986 (IC 95 %: 0.963–1.000)	Cumple
H1: superar la línea base en validación	F1 0.948 (SVM) y 0.921 (RL) frente a 0	Se sostiene
H2: τ que reduce el error con revisión ≤ 15 %	τ = 0.70: 1.35 % → 0.90 % con 0.63 % de revisión	Se sostiene (validación)
H3: metas cumplidas por el modelo congelado	Ambas metas en prueba	Se sostiene
Prototipo funcional	Clasifica, se abstiene, lista y guarda revisiones	Cumple (local)
Generalización a español, a fraudes actuales y a usuarios	Sin evaluar	Pendiente

Elaboración propia a partir de las tablas de la sección 4.3.

d) Recomendaciones técnicas

- **Ampliar la búsqueda de hiperparámetros y de representaciones.** Como ambos algoritmos eligieron C = 10, el límite de la rejilla, conviene evaluar C ∈ {10, 30, 100} y n-gramas de caracteres, que fueron la mejor representación en Şahin et al. (2026). Para no reutilizar la prueba actual, ya observada, la comparación debe hacerse con validación cruzada anidada sobre entrenamiento y validación, o con una nueva partición retenida. La recomendación se justificaría si mejora el recall sin bajar la precision por debajo de 0.95.
- **Complementar la abstención por confianza con un rechazo por novedad.** El 73 % de los errores de prueba (11 de 15) se aceptó con confianza ≥ 0.70, y siete con ≥ 0.98; los umbrales evaluados hasta 0.95 no capturaron siete de esos falsos negativos. Se propone derivar a revisión los mensajes fuera del dominio de entrenamiento —idioma distinto del inglés, muy pocos términos conocidos— y medir en validación cuántos errores confiados se recuperan y cuánta revisión adicional generan.
- **Auditar las etiquetas ambiguas y redactar una guía de revisión.** Varios falsos negativos son quejas sobre spam etiquetadas como spam. Antes de reentrenar con revisiones, el grupo debe definir criterios de etiquetado y medir el acuerdo entre revisores sobre una muestra.
- **Construir un conjunto en español con consentimiento antes de cualquier uso en el Perú.** El modelo actual sólo está validado en inglés; un conjunto local anonimizado permitiría estimar el desempeño real y recalibrar τ.

VI. EVIDENCIAS DE APRENDIZAJE Y USO RESPONSABLE DE IA

Esta sección distingue lo que puede acreditarse con evidencias de lo que el grupo debe completar. Gran parte del código, de la búsqueda bibliográfica y de la redacción se generó en una sesión asistida por IA (sección 6.3); esas actividades no se presentan como aprendizaje del grupo mientras sus integrantes no las revisen y expliquen.

6.1 Necesidades de aprendizaje identificadas

Tabla 23

Necesidades de aprendizaje identificadas a partir de los requisitos técnicos del proyecto

Concepto o herramienta	Dataset o métrica	Dificultad	Relación con el proyecto	Fuente
Partición agrupada y fuga de información	Duplicados del corpus; F1 de CV	**[POR COMPLETAR]**	Evitar evaluar con textos ya vistos	Kapoor y Narayanan (2023); Kaufman et al. (2012)
TF-IDF y n-gramas	F1 de spam	**[POR COMPLETAR]**	Representación de los mensajes	Salton y Buckley (1988)
RL y SVM lineal regularizadas	C; F1; curvas de aprendizaje	**[POR COMPLETAR]**	Algoritmos comparados	Cortes y Vapnik (1995); Fan et al. (2008)
Calibración de probabilidades	Brier; ECE; diagrama de fiabilidad	**[POR COMPLETAR]**	La confianza decide la revisión	Platt (2000); Silva Filho et al. (2023)
Clasificación selectiva	Cobertura; error entre aceptados	**[POR COMPLETAR]**	Núcleo de la pregunta de investigación	Chow (1970); Hendrickx et al. (2024)
Flask, SQLite y Joblib	Recorrido funcional	**[POR COMPLETAR]**	Integración en la aplicación	Pallets (s. f.); SQLite Consortium (s. f.); Joblib developers (s. f.)

Nota. La columna «Dificultad» debe registrarla cada integrante según su experiencia; no se infirió.

Elaboración propia.

6.2 Estrategias de aprendizaje aplicadas

Tabla 24

Estrategias aplicadas y acreditables, con indicación de quién las realizó

Estrategia	Actividad verificable	Evidencia	Realizada por
Revisión de *papers*	Búsqueda, lectura y extracción de 10 antecedentes y 4 marcos recientes; verificación de DOI; revalidación de indexación pendiente	Anexo A	Asistente de IA; lectura por el grupo **[POR COMPLETAR]**
Obtención de fuentes	Descarga del ZIP oficial de UCI y del PDF de Abdel-Jaber (2025)	SHA-256 del ZIP; PDF en `Bibliografia/`	Grupo (18-10-2026)
Lectura técnica	Documentación de `StratifiedGroupKFold` y `CalibratedClassifierCV` aplicada al código	`src/spam_sms/`	Asistente de IA; grupo **[POR COMPLETAR]**
Experimentación	Tres ejecuciones completas del notebook con métricas de prueba idénticas	`results/`; notebook ejecutado	Asistente de IA
Revisión de código	Pruebas unitarias; corrección de una expectativa errónea de prueba	`results/pytest_salida.txt`	Asistente de IA; revisión por el grupo **[POR COMPLETAR]**

Elaboración propia.

6.3 Uso responsable de IA generativa y aprendizaje permanente

Tabla 25
Registro de uso de IA generativa

Herramienta	Prompt o consulta	Respuesta obtenida	Verificación o fuente contrastada	Decisión final	Evidencia de comprensión
Claude (Anthropic); sesión configurada con el modelo claude-opus-5-5	Instrucción del grupo del 18-10-2026 para desarrollar el artículo, el notebook y el 50 % del código (Anexo H)	Borrador del artículo; código; notebook ejecutado; prototipo; figuras; búsqueda bibliográfica	Métricas trazadas a results/; DOI en Crossref y editoriales; indexación en SJR; PDF aportado por el grupo	**[POR COMPLETAR por el grupo]**	**[POR COMPLETAR: explicación propia de cada integrante]**
Claude (misma sesión)	Acceso al dataset: la IA no pudo descargar desde UCI por política de red y pidió el ZIP al grupo	Instrucciones de descarga sin descomprimir	ZIP oficial con SHA-256 registrado; readme v.1	Usar el ZIP oficial	Registro de la sesión
[POR COMPLETAR]	Otras herramientas o consultas del grupo	—	—	—	—

Nota. El modelo que atendió cada respuesta puede diferir del configurado. Ninguna cifra del artículo procede de la IA sin un archivo de resultados que la respalde.

Elaboración propia.

6.4 Mejoras realizadas en el proyecto

Tabla 26
Mejoras realizadas con evidencia

Conocimiento aplicado	Sección	Cambio antes / después	Evidencia	Efecto en métricas
Lectura sin comillado del TSV	3.5	5572 filas (comillado por defecto) → 5574 (QUOTE_NONE)	diagnostico_dataset.json	No medido: no se entrenó con la versión defectuosa
Agrupación de duplicados	3.4; 4.3	CV estratificada → CV estratificada y agrupada	diagnostico_fuga_cv.csv	F1 CV: RL 0.944 → 0.948; SVM 0.960 → 0.960 (sin cambio apreciable)
Verificación de la clase positiva	1.1; 5.1	Antecedente «pendiente» → métricas de ham (inferido del PDF)	PDF de Abdel-Jaber (2025)	Evita una comparación errónea; no afecta métricas propias
Revisión de pruebas	3.6	1 prueba fallida por expectativa errónea → 4 superadas	pytest_salida.txt	Ninguno

Elaboración propia.

VII. CONSIDERACIONES ÉTICAS RELACIONADAS CON IA

La Tabla 27 presenta la matriz de riesgos éticos. Salvo los falsos positivos y negativos, cuyas frecuencias se observaron en la partición de prueba, las valoraciones de probabilidad y severidad son estimaciones cualitativas del equipo y deberán revisarse con datos de uso. El prototipo no debe recibir mensajes reales de terceros mientras no se implementen las mitigaciones de privacidad y retención.

Tabla 27
Matriz de riesgos éticos relacionados con IA

Riesgo ético o fuente	Descripción e impacto	Probabilidad / severidad	Grupo afectado	Acción de mitigación
Privacidad	La aplicación almacena el texto completo, que puede contener nombres, teléfonos o datos bancarios	Alta / alta (estimación)	Usuarios y terceros mencionados	Minimizar lo guardado; enmascarar números, enlaces y nombres; cifrar la base; acceso por rol; aviso previo
Conservación de mensajes	El prototipo no borra registros; la retención indefinida amplía el daño ante filtraciones	Alta / media (estimación)	Usuarios	Retención limitada (propuesta: 30 días) y borrado a solicitud; registrar quién accede
Datos personales del corpus	Los mensajes públicos contienen números y apodos	Media / media (estimación)	Personas mencionadas	Usar sólo para investigación; mostrar extractos truncados; respetar la licencia CC BY 4.0
Falsos bloqueos (FP)	Un mensaje legítimo marcado como spam puede perderse	Observada: 2 de 965 ham (0.21 %) / alta	Remitentes y destinatarios legítimos	No borrar, sólo etiquetar; meta de precision ≥ 0.95; derivar casos dudosos; vigilar los FP
Falsos negativos	Spam no detectado expone al usuario al fraude	Observada: 13 de 150 spam (8.7 %) / alta	Usuarios	Advertir que el sistema no garantiza protección; educación sobre *smishing*
Sesgo lingüístico	Corpus en inglés británico y singapurense de 2011; peor desempeño esperable en español y mezclas	Alta / alta (estimación)	Hablantes de español y multilingües	Aviso de alcance en la interfaz; derivar a revisión los mensajes en otro idioma; datos locales
Confianza excesiva	Usuarios o revisores pueden aceptar la decisión automática sin crítica	Media / media (estimación)	Usuarios y revisores	Mostrar probabilidad y límites; capacitación; no presentarla como verificación definitiva
Errores de revisión humana	El revisor puede etiquetar mal o con criterios inconsistentes	Media / media (no medida)	Usuarios; calidad de datos futuros	Guía de etiquetado; doble revisión de una muestra; medir el acuerdo
Integridad académica	Contenido generado con IA presentado como trabajo propio	Media / alta (estimación)	Grupo; evaluación del curso	Declaración en la sección 6.3; revisión y explicación propia del código por cada integrante

Elaboración propia.

VIII. ACTIVIDADES ADICIONALES, REFERENCIAS Y ANEXOS

8.1 Actividades adicionales

Este espacio se reserva para certificaciones o cursos externos (IBM, Microsoft, AWS u otros) relacionados con el proyecto. A la fecha no se ha acreditado ninguna; sólo se incorporarán las que cuenten con constancia verificable.

Tabla 28
Actividades y certificaciones externas

Integrante	Actividad o certificación	Entidad	Fecha	Evidencia (enlace o código)
[POR COMPLETAR]	[POR COMPLETAR]	—	—	—

Elaboración propia.

8.2 Referencias

Abdel-Jaber, H. (2025). Detecting spam and ham SMS messages using natural language processing and machine learning algorithms. *PeerJ Computer Science, 11*, Artículo e3232. https://doi.org/10.7717/peerj-cs.3232

Almeida, T. A., Gómez Hidalgo, J. M. y Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. En *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259–262). ACM. https://doi.org/10.1145/2034691.2034742

Almeida, T. y Hidalgo, J. (2011). *SMS Spam Collection* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

Altunay, H. C. y Albayrak, Z. (2024). SMS spam detection system based on deep learning architectures for Turkish and English messages. *Applied Sciences, 14*(24), Artículo 11804. https://doi.org/10.3390/app142411804

Bani Younes, M. y Ababneh, A. (2026). Machine learning based spam detection in digital communication systems: A comparative analysis. *Systems, 14*(3), Artículo 229. https://doi.org/10.3390/systems14030229

Calero Sinche, D. F., Acuña Meléndez, M. y Ovalle, C. (2024). Application of classification algorithms for smishing detection on mobile devices: Literature review. *IAES International Journal of Artificial Intelligence, 13*(4), 3750–3760. https://doi.org/10.11591/ijai.v13.i4.pp3750-3760

Castañeda Chávez, R. A. (2013). *Propuesta de diseño de sistema anti-spam para SMS aplicado a Movistar Perú* [Tesis de bachiller, Universidad Nacional de Trujillo]. Repositorio Institucional UNITRU. https://hdl.handle.net/20.500.14414/8551

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. y Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory, 16*(1), 41–46. https://doi.org/10.1109/TIT.1970.1054406

Cortes, C. y Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Diario Oficial El Peruano. (2023, 26 de mayo). *Cuidado con las estafas vía mensajes de texto: ¿Sabe cómo protegerse de este tipo de fraude?* https://www.elperuano.pe/noticia/213777-cuidado-con-las-estafas-via-mensajes-de-texto-sabe-como-protegerse-de-este-tipo-de-fraude

Diario Oficial El Peruano. (2024, 15 de abril). *¡Cuidado! Crecen denuncias de fraude informático en el Perú.* https://elperuano.pe/noticia/241353-cuidado-crecen-denuncias-de-fraude-informatico-en-el-peru

Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R. y Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research, 9*, 1871–1874. https://www.jmlr.org/papers/v9/fan08a.html

Federal Trade Commission. (2025, 14 de abril). *Top text scams of 2024* [Data Spotlight]. https://www.ftc.gov/news-events/data-visualizations/data-spotlight/2025/04/top-text-scams-2024

Ghourabi, A. y Alohaly, M. (2023). Enhancing spam message classification and detection using transformer-based embedding and ensemble learning. *Sensors, 23*(8), Artículo 3861. https://doi.org/10.3390/s23083861

Hendrickx, K., Perini, L., Van der Plas, D., Meert, W. y Davis, J. (2024). Machine learning with a reject option: A survey. *Machine Learning, 113*(5), 3073–3110. https://doi.org/10.1007/s10994-024-06534-x

Joblib developers. (s. f.). *Joblib: Running Python functions as pipeline jobs* (Versión 1.6.0) [Software]. https://joblib.readthedocs.io/

Kapoor, S. y Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns, 4*(9), Artículo 100804. https://doi.org/10.1016/j.patter.2023.100804

Kaufman, S., Rosset, S., Perlich, C. y Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), 1–21. https://doi.org/10.1145/2382577.2382579

Lulli Caceres, J. A. y Ortiz Gutierrez, J. L. (2024). *El artículo 8 de la Ley de Delitos Informáticos en el tratamiento de los casos de fraude informático en la Segunda Fiscalía Penal Corporativa de Trujillo - 2023* [Tesis de licenciatura, Universidad Privada del Norte]. Repositorio Institucional UPN. https://hdl.handle.net/11537/40109

Maqsood, U., Ur Rehman, S., Ali, T., Mahmood, K., Alsaedi, T. y Kundi, M. (2023). An intelligent framework based on deep learning for SMS and e-mail spam detection. *Applied Computational Intelligence and Soft Computing, 2023*, Artículo 6648970. https://doi.org/10.1155/2023/6648970

Mosqueira-Rey, E., Hernández-Pereira, E., Alonso-Ríos, D., Bobes-Bascarán, J. y Fernández-Leal, Á. (2023). Human-in-the-loop machine learning: A state of the art. *Artificial Intelligence Review, 56*(4), 3005–3054. https://doi.org/10.1007/s10462-022-10246-w

Niculescu-Mizil, A. y Caruana, R. (2005). Predicting good probabilities with supervised learning. En *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

Pallets. (s. f.). *Flask* (Versión 3.1.3) [Software]. https://flask.palletsprojects.com/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Platt, J. C. (2000). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. En A. J. Smola, P. L. Bartlett, B. Schölkopf y D. Schuurmans (Eds.), *Advances in large-margin classifiers* (pp. 61–74). MIT Press.

Redacción Correo. (2011, 2 de junio). Trujillo: Continúan estafas y extorsiones por celular. *Diario Correo*. https://diariocorreo.pe/peru/trujillo-continuan-estafas-y-extorsiones-por-celular-486685/

Reyes, Y., Casallo, D., Mansilla, J., Escarcena, M. y Buleje, M. (2026). Predictive model of phishing attacks using machine learning for fintech companies in Peru. En T. Guarda, F. Portela y M. F. Augusto (Eds.), *Advanced research in technologies, information, innovation and sustainability: ARTIIS 2025, International Workshops, Cartagena de Indias, Colombia, October 21–23, 2025, Revised Selected Papers, Part I* (Communications in Computer and Information Science, Vol. 2791, pp. 233–244). Springer. https://doi.org/10.1007/978-3-032-16848-1_18

Şahin, M. S., Şahin, D. Ö. y Salah, A. F. (2026). Revisiting SMS spam detection: The impact of feature representation on classical machine learning models. *Electronics, 15*(4), Artículo 894. https://doi.org/10.3390/electronics15040894

Salman, M., Ikram, M. y Kaafar, M. A. (2024). Investigating evasive techniques in SMS spam filtering: A comparative analysis of machine learning models. *IEEE Access, 12*, 24306–24324. https://doi.org/10.1109/ACCESS.2024.3364671

Salton, G. y Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Silva Filho, T., Song, H., Perello-Nieto, M., Santos-Rodriguez, R., Kull, M. y Flach, P. (2023). Classifier calibration: A survey on how to assess and improve predicted class probabilities. *Machine Learning, 112*(9), 3211–3260. https://doi.org/10.1007/s10994-023-06336-7

SQLite Consortium. (s. f.). *SQLite* (Versión 3.45.1) [Software]. https://www.sqlite.org/

The pandas development team. (s. f.). *pandas* (Versión 3.0.5) [Software]. https://pandas.pydata.org/

8.3 Anexos

Anexo A. Matriz bibliográfica, registro de búsqueda y evidencia de indexación

Tabla 29

Registro de búsqueda bibliográfica (18 de octubre de 2026)

Fuente	Términos o consulta	Resultado relevante	Decisión
Buscador web académico	«SMS spam detection» + «SMS Spam Collection» + TF-IDF, SVM, logistic regression, 2022–2026	Ghourabi y Alohaly; Şahin et al.; Bani Younes y Ababneh; Abdel-Jaber; Altunay y Albayrak	Incluidos (internacionales)
Buscador web y DOAJ	Salman, Ikram, Kaafar; Maqsood et al.	Artículos de IEEE Access y ACISC	Incluidos; PDF primario de Salman et al. leído y Tablas 4, 12 y 14 verificadas visualmente
ALICIA (CONCYTEC)	detección spam SMS aprendizaje automático; smishing; Perú	Calero Sinche et al. (2024, UTP)	Incluido (nacional)
Buscador web	phishing machine learning Perú	Reyes et al. (2026, UPC); Interfases (U. de Lima, 2020)	Reyes: incluido (congreso); Interfases: excluido por fecha
ALICIA, buscador web, RENATI y repositorios institucionales	spam, SMS, smishing, phishing + Trujillo/La Libertad; búsquedas por dominio RENATI, UNT, UPAO, UCV y UPN (18-10-2026)	Lulli Caceres y Ortiz Gutierrez (2024, UPN-Trujillo); Castañeda Chávez (2013, UNT)	UPN 2024 incluido como contexto jurídico local; UNT 2013 excluido por fecha. Clasificador local pendiente; accesos limitados en RENATI, UNT, UPAO y UCV
Crossref, editoriales y Scimago	Verificación de DOI, autores, volumen, páginas e indexación	Metadatos confirmados; indexación pendiente de revalidación	Registro en Tabla 31; conservar fichas de Scimago [PENDIENTE]

Elaboración propia.

Tabla 30

Matriz de extracción de antecedentes

Antecedente y título	Objetivo general	Metodología	Población	Muestra	Resultados numéricos (fuente)
Ghourabi y Alohaly (2023). *Enhancing spam message classification...*	Proponer un detector con *embeddings* de transformadores y conjuntos	GPT-3 (1536 dim.) frente a TF-IDF, *word embedding* y BERT; votación ponderada con optimización bayesiana; 80/20; Wilcoxon	SMS en inglés	5574 (4827 ham; 747 spam)	Conjunto: Acc 0.9991; P 1.000; R 0.9932; F1 0.9966 (Tabla 3). SVM + TF-IDF: Acc 0.9785 (Tabla 2)
Salman et al. (2024). *Investigating evasive techniques...*	Caracterizar la evolución del spam y evaluar modelos y servicios frente a la evasión	Depuración y etiquetado; representaciones léxicas/semánticas; partición 80/20 y conjunto separado de 225 SMS spam para evasión	SMS de fuentes públicas y voluntarios	67 018: 40 837 ham; 26 181 spam (Tabla 4; discrepante con el resumen)	TCSVM + Word2Vec: P 99 %, R 98 %, Acc 97.8 %, F1 99 % (Tabla 12, p. 24315). Conjunto adversarial separado: Acc 83.1 %/F1 91 % sin ataque y Acc 11.6 %/F1 21 % con espacios (Tabla 14, p. 24317)
Altunay y Albayrak (2024). *SMS spam detection system based on deep learning...*	Proponer CNN + GRU para turco e inglés	Limpieza, normalización, *embedding* propio; CNN, GRU, CNN + GRU	SMS en turco e inglés	10 325; prueba: 965 no spam y 665 spam	Acc 0.9908; P 0.9907; R 0.9938; F1 0.9922 (sección 5); 956/965 y 659/665 correctos (Figura 6)

Tabla 30 (continuación)

Antecedente y título	Objetivo general	Metodología	Población	Muestra	Resultados numéricos (fuente)
Maqsood et al. (2023). *An intelligent framework based on deep learning...*	Comparar ML y CNN para spam en SMS y correo	Tokenización, *stemming*; BoW y TF-IDF; SVM, NB, RF, CNN; ~67.7/32.3	SMS y correos	SMS: 5000 (750 spam)	SVM: Acc 99.6 % en SMS (secciones 4.4 y 5); P, R y F1 en la Tabla 4, no extraídas
Abdel-Jaber (2025). *Detecting spam and ham SMS messages...*	Detectar SMS spam y ham con PLN y ML	CountVectorizer; *stemming* o lematización; 10 particiones estratificadas; sobremuestreo; KNN, DT, RF, GB, MLP, SVM	SMS en inglés	5572 (747 spam; 4825 ham)	Desbalanceado, *stemming*: SVM Acc 0.9740, P 0.9709, R 1.000, F1 0.9852; MLP F1 0.9923 (Tablas 7–10); ham como positiva
Şahin et al. (2026). *Revisiting SMS spam detection...*	Medir el efecto de la representación en clasificadores clásicos	7 representaciones × NB, SVM lineal, RF, KNN, RL; CV estratificada de 10 pliegues; sin balanceo	SMS en inglés	5574 (747 spam)	Trigramas de caracteres + RL: Acc 98.55 %; P 98.55 %; R 90.50 %; F1 94.32 %; AUC 0.9893 (Tabla 5); 10 FP y 71 FN (Figura 4)
Bani Younes y Ababneh (2026). *Machine learning based spam detection...*	Comparar modelos básicos y su generalización entre conjuntos	TF-IDF (500 términos); eliminación de duplicados; SMOTE en entrenamiento; prueba entre dominios	Correo, SMS y redes sociales	SMS: 5100 (UCI) + Kaggle externo	SMS mismo dominio: Acc ≈ 85–96 %, P/R ≈ 75–85 %; entre dominios: Acc ≈ 75–83 %, P/R ≈ 45–65 % (Figura 6)
Calero Sinche et al. (2024). *Application of classification algorithms for smishing...*	Identificar técnicas de clasificación para detectar *smishing*	Revisión de literatura en Scopus y ARDI	Estudios publicados	184 recuperados; 36 incluidos	RF, árboles y redes neuronales como los más usados; métricas: no aplica
Reyes et al. (2026). *Predictive model of phishing attacks...*	Predecir correos de *phishing* en *fintech*	XGBoost + SMOTE (actas de congreso)	Correos	No reportada en la página consultada	No verificado (texto completo no accesible)
Lulli Caceres y Ortiz Gutierrez (2024). El artículo 8 de la Ley de Delitos Informáticos...	Evaluar la aplicación del artículo 8 en los casos de fraude informático de la Segunda Fiscalía Penal Corporativa de Trujillo durante 2023	Enfoque cualitativo; básica; diseño no experimental; análisis documental y entrevistas (pp. 25–29)	Carpetas fiscales archivadas en investigación preliminar y fiscales especializados; tamaño total no reportado (pp. 26–27)	11 carpetas fiscales y 3 fiscales entrevistados; selección intencional (p. 27)	Dificultades para calificar el delito, individualizar responsables y tratar evidencia digital (pp. 67–72). Tabla 14, p. 51: caso iniciado por SMS. Sin métricas de clasificación ni prevalencia poblacional

Elaboración propia a partir de los artículos originales.

Tabla 31

Registro de revistas de los doce artículos recientes y estado de verificación de su indexación

Artículo	Revista (ISSN)	Cobertura declarada*	Cuartil 2025 declarado*	Uso en el artículo
Ghourabi y Alohaly (2023)	Sensors (1424-8220)	2001–2026	Q1	Antecedente internacional
Salman et al. (2024)	IEEE Access (2169-3536)	2013–2026	Q1	Antecedente internacional
Altunay y Albayrak (2024)	Applied Sciences (2076-3417)	2011–2026	Q2	Antecedente internacional
Maqsood et al. (2023)	Applied Computational Intelligence and Soft Computing (1687-9724)	2016–2026	Q1	Antecedente internacional
Abdel-Jaber (2025)	PeerJ Computer Science (2376-5992)	2015–2025	Q2	Antecedente internacional
Şahin et al. (2026)	Electronics (2079-9292)	2012–2026	Q2	Antecedente internacional
Bani Younes y Ababneh (2026)	Systems (2079-8954)	2013–2026	Q2	Antecedente internacional
Calero Sinche et al. (2024)	IAES Int. Journal of Artificial Intelligence (2252-8938)	2018–2025	Q3	Antecedente nacional
Hendrickx et al. (2024)	Machine Learning (0885-6125)	1986–2026	Q1	Marco teórico
Silva Filho et al. (2023)	Machine Learning (0885-6125)	1986–2026	Q1	Marco teórico
Mosqueira-Rey et al. (2023)	Artificial Intelligence Review (0269-2821)	1986–2026	Q1	Marco teórico
Kapoor y Narayanan (2023)	Patterns (2666-3899)	2020–2026	Q1	Marco teórico

Nota. Se identificaron doce artículos de revista publicados entre 2023 y 2026, con metadatos comprobados en Crossref. *La cobertura y los cuartiles declarados se conservan de la versión anterior y no pudieron revalidarse porque Scimago restringió el acceso. [PENDIENTE: conservar las fichas individuales con URL, ISSN, año de cobertura y cuartil]. El cumplimiento del mínimo de artículos indexados en Scopus queda sujeto a esa comprobación. La indexación en Web of Science no se verificó.

Elaboración propia a partir del registro anterior de Scimago y de los metadatos de Crossref; véase la limitación de verificación en la nota.

Anexo B. Archivos de datos derivados

Tabla 32

Archivos generados por el notebook y su contenido

Archivo (en codigo/results/)	Contenido
entorno_ejecucion.json	Fecha, versión de Python y bibliotecas, plataforma y semilla
diagnostico_dataset.json	Conteos por clase, vacíos, duplicados, contradicciones, longitudes, SHA-256 del ZIP
particiones.csv / tablas/particiones_resumen.csv	Partición asignada a cada mensaje / resumen por partición
tablas/cv_rejilla.csv / mejores_hiperparametros.json	Resultados de las 12 configuraciones / configuración elegida por algoritmo
tablas/diagnostico_fuga_cv.csv	CV agrupada frente a no agrupada
tablas/curva_aprendizaje_*.csv	Curvas de aprendizaje
tablas/calibracion_validacion.csv / fiabilidad_validacion_*.csv	Brier, pérdida logarítmica, ECE / datos del diagrama de fiabilidad
tablas/comparacion_validacion.csv / umbrales_validacion_*.csv	Comparación de modelos / métricas selectivas por τ
decisiones_congeladas.json	Modelo, hiperparámetros, τ, metas y SHA-256 del artefacto
evaluacion_final_prueba.json / tablas/umbrales_prueba_descriptivo.csv	Métricas finales, IC, selectivas, revisión simulada, sensibilidad / τ en prueba
tablas/errores_prueba.csv / sondas_exploratorias.csv	Errores de prueba / sondas de demostración
latencia_inferencia.json / pytest_salida.txt	Tiempos de ajuste e inferencia / resultado de las pruebas

Elaboración propia.

Anexo C. Configuraciones congeladas

Listado 5

Contenido de decisiones_congeladas.json y entorno_ejecucion.json (extracto)

```
# decisiones_congeladas.json
{
 "fecha_congelamiento": "2026-10-18T19:50:08-05:00",
 "modelo_elegido": "linsvm",
 "motivo": "mayor F1 de spam en validación",
 "hiperparametros": {
  "C": 10.0,
  "ngram_range": [1, 2],
  "f1_cv": 0.9597
 },
 "calibracion": "sigmoide (Platt), CV agrupada 5 pliegues en entrenamiento",
 "tau": 0.7,
 "tau_factible_en_validacion": true,
 "criterio_tau": "max cobertura con error_aceptados<=0.010 y tasa_revision<=0.15 (validación)",
 "metas_preregistradas_prueba": {
  "f1_spam_min": 0.9,
  "precision_spam_min": 0.95
 },
 "ruta_modelo": "models/modelo_spam_v1.joblib",
 "sha256_modelo": "de87978042e6c821cb8a4fb2dff3cee4ada58ac3ff1ef65ef0ea703e55482705"
}

# entorno_ejecucion.json
{
 "fecha_ejecucion": "2026-10-18T19:49:30-05:00",
 "python": "3.13.16",
 "plataforma": "Linux-6.18.44-fc-v139-x86_64-with-glibc2.39",
 "procesador": "x86_64",
 "nucleos_logicos": 2,
 "numpy": "2.5.3",
 "pandas": "3.0.5",
 "scikit-learn": "1.9.1",
 "joblib": "1.6.0",
 "matplotlib": "3.11.2",
 "semilla": 42
}
```

Archivos de codigo/results/ generados en la última ejecución del notebook.

Anexo D. Protocolo experimental: correspondencia con el protocolo propuesto

Tabla 33

Correspondencia entre el protocolo propuesto y la implementación

Paso propuesto	Implementación	Estado
1. Inspeccionar versión, clases, vacíos, duplicados y contradicciones	`diagnose()` sobre el ZIP oficial; SHA-256; detección del error de comillado	Hecho
2. Duplicados sin cruzar particiones	Grupos por texto normalizado; verificación de 0 grupos compartidos	Hecho
3. Partición estratificada 60/20/20, semilla 42	`StratifiedGroupKFold` (5 y luego 4 pliegues): 59.99/20.00/20.00	Hecho
4. *Pipelines* TF-IDF + clasificador	`Pipeline` de scikit-learn	Hecho
5. RL y SVM en las mismas condiciones	Mismos pliegues, vectorizador y semilla	Hecho
6. C en {0.1, 1, 10}; unigramas frente a uni + bigramas	Rejilla completa de 12 configuraciones	Hecho; óptimo en el límite
7. CV estratificada dentro del entrenamiento	CV estratificada **y agrupada** de 5 pliegues	Hecho (más estricto)
8. Calibración sin usar la prueba ni reutilizar datos de ajuste	Platt con ajuste cruzado en 5 pliegues; el modelo final promedia 5 SVM entrenadas con el 80 % del entrenamiento cada una	Hecho; diferencia documentada
9. Selección de modelo y regla en validación	Reglas preregistradas en el código	Hecho
10. Congelar y evaluar	`decisiones_congeladas.json` antes de la prueba; 3 ejecuciones idénticas	Hecho
11. Persistir e integrar en la aplicación	Joblib + Flask; misma función `decide`	Hecho (local)
12. Documentar errores, limitaciones y estado	Secciones 4.3, 5.3 y Tabla 11	Hecho

Elaboración propia.

Anexo E. Código ampliado

Listado 6

Partición estratificada y agrupada (src/spam_sms/data.py)

```
def group_key(text: str) -> str:
    return re.sub(r"\s+", " ", text.strip().lower())

def grouped_stratified_split(df, seed=42):
    groups = df["text"].map(group_key).to_numpy()
    y = df["y"].to_numpy(); idx = np.arange(len(df))
    outer = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=seed)
    rest, test = next(outer.split(idx, y, groups))              # ~20 % prueba
    inner = StratifiedGroupKFold(n_splits=4, shuffle=True, random_state=seed)
    tr_rel, va_rel = next(inner.split(rest, y[rest], groups[rest]))  # ~20 % validación
    train, val = rest[tr_rel], rest[va_rel]
    g_tr, g_va, g_te = set(groups[train]), set(groups[val]), set(groups[test])
    assert not (g_tr & g_va) and not (g_tr & g_te) and not (g_va & g_te)
    return np.sort(train), np.sort(val), np.sort(test)
```

Código ejecutado; extracto sin docstrings.

Listado 7

Métricas selectivas y regla de elección de τ (src/spam_sms/decision.py)

```
def selective_metrics(y_true, p_spam, tau):
  y_hat, acc = decide(p_spam, tau); n = len(y_true); n_acc = int(acc.sum())
  out = {"tau": tau, "n_aceptados": n_acc, "n_revision": n - n_acc,
      "cobertura": n_acc / n, "tasa_revision": (n - n_acc) / n,
      "errores_aceptados": int(((y_hat != y_true) & acc).sum())}
  out["error_aceptados"] = out["errores_aceptados"] / n_acc if n_acc else float("nan")
  ...                          # métricas por clase del subconjunto aceptado
  return out

def select_tau(rows, max_error=0.010, max_review=0.15):
  feas = [r for r in rows if r["error_aceptados"] <= max_error and r["tasa_revision"] <= max_review]
  if feas:
    return {"tau": sorted(feas, key=lambda r: (-r["cobertura"], r["tau"]))[0]["tau"], "factible": True}
  cand = [r for r in rows if r["tasa_revision"] <= max_review]
  return {"tau": sorted(cand, key=lambda r: (r["error_aceptados"], r["tau"]))[0]["tau"], "factible": False}
```

Código ejecutado; extracto simplificado.

Listado 8

Rutas de bandeja y revisión (app/app.py)

```
@app.route("/pendientes")
def pendientes():
  filas = db().execute("SELECT m.* FROM mensajes m LEFT JOIN revisiones r ON r.mensaje_id = m.id "
            "WHERE m.decision = ? AND r.id IS NULL ORDER BY m.id", (REVIEW,)).fetchall()
  return render_template("pendientes.html", filas=filas, tau=TAU)

@app.route("/revisar/<int:mensaje_id>", methods=["POST"])
def revisar(mensaje_id):
  etiqueta = request.form.get("etiqueta")          # "0" = ham, "1" = spam
  revisor = (request.form.get("revisor") or "").strip()[:60] or "revisor_sin_nombre"
  if etiqueta not in {"0", "1"}:
    return redirect(url_for("pendientes"))
  db().execute("INSERT OR IGNORE INTO revisiones (mensaje_id, etiqueta_revisor, revisor, "
         "comentario, revisado_en) VALUES (?, ?, ?, ?, ?)", (mensaje_id, int(etiqueta), revisor, ...))
  db().commit()                                    # no reentrena el modelo
  return redirect(url_for("revisiones"))
```

Código ejecutado; extracto simplificado.

Anexo F. Repositorio, estructura e instrucciones de ejecución

Repositorio remoto: **[PENDIENTE: REPOSITORIO]**. Mientras se crea, el código se entrega en la carpeta del proyecto con la estructura del Listado 9. Se recomienda publicar el repositorio con el historial de contribuciones de cada integrante para acreditar el avance.

Listado 9

Estructura de la carpeta del proyecto y comandos de ejecución

```
Proyecto del Curso/
├── Dataset/sms+spam+collection.zip      ZIP oficial de UCI (no modificar)
├── Bibliografia/AbdelJaber_2025_PeerJCS.pdf
├── Articulo/                        artículo (.docx, .pdf), figuras y nota de pendientes
└── codigo/
  ├── README.md · requirements.txt
  ├── src/spam_sms/   data.py · modeling.py · decision.py
  ├── notebooks/     01_experimento_spam_sms.ipynb · build_notebook.py
  ├── app/          app.py · templates/ · static/
  ├── models/        modelo_spam_v1.joblib
  ├── results/      *.json · tablas/ · figuras/ · capturas/
  └── tests/        test_nucleo.py · capturas_app.py

# Windows (PowerShell), desde Proyecto del Curso\codigo
py -3.13 -m venv .venv ; .venv\Scripts\Activate.ps1
pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace notebooks/01_experimento_spam_sms.ipynb
python -m pytest -q tests
python app/app.py        # abrir http://127.0.0.1:5000
```

Elaboración propia.

Anexo G. Evidencias de ejecución

La ejecución documentada produjo las huellas de la Tabla 34. Si el grupo vuelve a ejecutar el notebook con las mismas versiones, el SHA-256 del ZIP debe coincidir; el del modelo debería coincidir si se usan las mismas versiones de las bibliotecas. La Figura 13 completa las capturas de la sección 4.2.

Tabla 34

Huellas y resultados de verificación de la ejecución documentada

Elemento	Valor
ZIP del corpus (SHA-256)	1587ea43e58e82b14ff1f5425c88e17f8496bfcdb67a583dbff9eefaf9963ce3
SMSSpamCollection (SHA-256)	7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d
modelo_spam_v1.joblib (SHA-256)	de87978042e6c821cb8a4fb2dff3cee4ada58ac3ff1ef65ef0ea703e55482705
Notebook	43 celdas (26 de código), todas ejecutadas; última ejecución: 18-10-2026
Pruebas unitarias	4 superadas (pytest 9.1.1)
Recorrido de la aplicación	3 mensajes clasificados (spam, legítimo, pendiente); 1 revisión registrada

Elaboración propia.

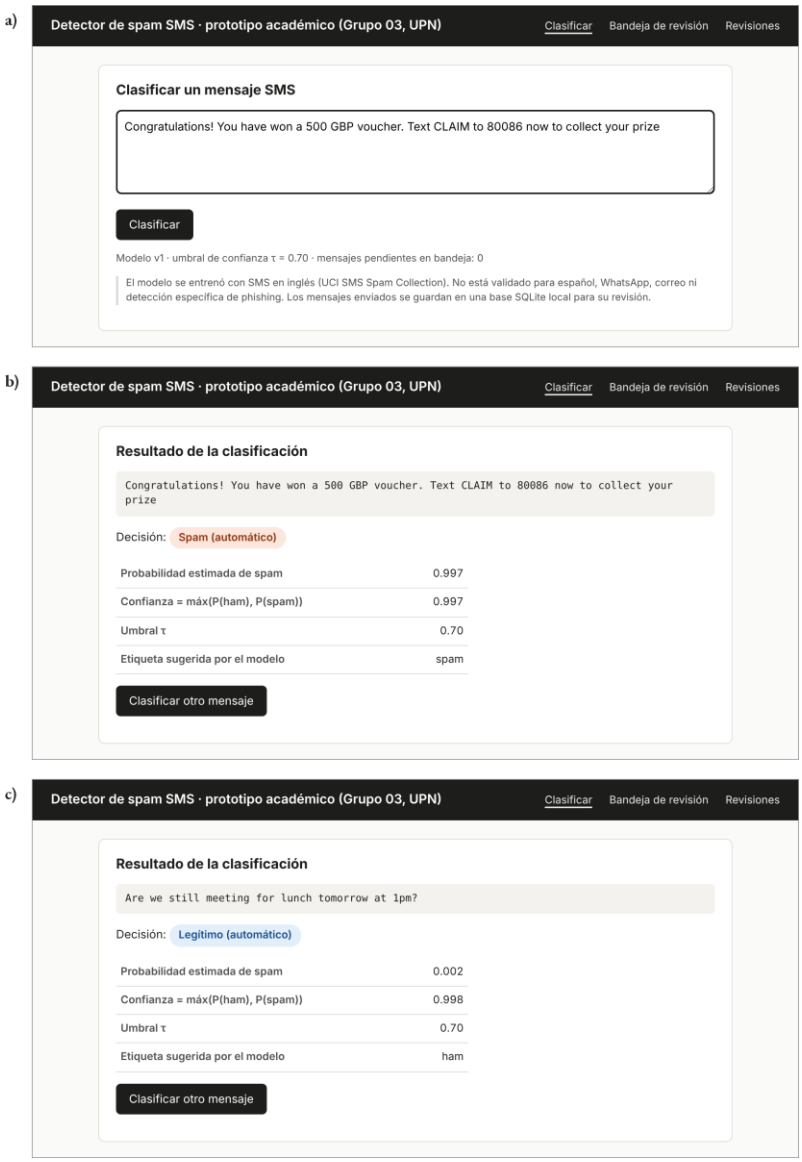


Figura 13

Formulario con un mensaje de demostración (a) y decisiones automáticas spam (b) y legítimo (c)

Capturas de la aplicación Flask en ejecución local (18-10-2026), obtenidas con Playwright; mensajes de demostración.

Anexo H. Registros de uso de IA

El registro completo de la instrucción entregada al asistente de IA el 18 de octubre de 2026 se guarda en Articulo/anexos/registro_uso_IA.md, junto con la secuencia de solicitudes intermedias (descarga del ZIP oficial y del PDF de Abdel-Jaber). Cada integrante debe añadir sus propias consultas a herramientas de IA, con fecha, propósito y verificación **[POR COMPLETAR]**. La IA no tuvo acceso a datos personales de los integrantes; sus correos y ORCID se dejaron sin completar deliberadamente.

La continuación editorial con Codex (OpenAI), incluida la revisión visual, la maquetación y la verificación bibliográfica adicional, se registra en el mismo archivo de uso de IA. No se ejecutaron nuevos experimentos ni se seleccionaron parámetros con la prueba ya observada. La comprensión y validación final por cada integrante siguen pendientes [POR COMPLETAR].